VIBE CODING 教學案例 03

開窯預約
有帳號與資料的網站

哪些事情不能只相信前端。

有人繞過網頁直接送請求時，擋得住的只有資料庫的規則。

成品　台北大稻埕柴燒陶藝工作室「器 VESSEL」的開窯預約系統，上線後掛在品牌官網底下（例如 vessel.tw/kiln），案例一做的就是那個官網。任何人都看得到場次與剩餘名額；用 email 免密碼登入之後可以預約與取消，而且只看得到自己的那幾筆。

使用技術　Astro · Supabase（雲端）· 原生 CSS。不需要 Docker，也不用寫後端

時間　閱讀約 2.5 小時，自己動手約 7.8 小時

對象　沒有程式背景，不需要看懂 SQL 或 JavaScript

完成等級　階段 12 = Learning Prototype（有已知漏洞） / 階段 16 = Functional Prototype

核心　打包進網頁的金鑰本來就公開，擋住別人的是資料庫的規則

案例三：開窯預約

哪些事情不能只相信前端

⚠️ 開始之前一定要看：這個案例只能用練習用的資料

這個案例會動到真的資料庫，所以只能在練習用的環境做。

只能用	說明
新建的 Supabase 專案	專門為這個案例開一個新的，做完可以整個刪掉
假資料	階段 7 產生的五筆示範窯次。不要放任何真實的客戶、訂單、名單
測試用的帳號	你自己的信箱，或 Gmail 別名（見階段 12）。不要用同事或客戶的信箱

這些環境**不可以**拿來練習：公司正式資料、客戶資料、Production 的 Supabase 專案。任何**正在服務真實使用者**的環境也不行。

原因很具體。這個案例會反覆建表改權限、灌示範資料，還會**把某一窯的名額直接改到只剩 1**。階段 13 更會**故意重現一個超額預約的漏洞**。這些操作在練習專案裡沒有代價。在正式環境會造成真實損害，而且有些改不回來。

> **這個案例後面還有三個安全警告，出現時同樣不能跳過：** ① 不能外流的金鑰 `service_role` / `sb_secret`（階段 3、9、17） ② `.env` 絕對不能進版本控制（階段 9、17） ③ 本機測試用的 `_local-auth-stub.sql` **只能在自己電腦執行**。貼到 Supabase 會蓋掉真的帳號系統（階段 8）

> 這份手冊是寫給沒有程式背景的人的。
>
> 你不需要會寫 SQL，也不需要會寫 JavaScript。手冊本身幾乎不印 SQL。 SQL 是 AI 寫進檔案裡的，而你會要求它在每一段上面寫中文註解。你只讀那些註解。
>
> 這份手冊的重點是判斷。這件事該不該做、AI 給的東西對不對、畫面出問題時要往哪一層去找。

> **名詞**：SQL 是跟資料庫下指令用的語言。
> 你需要知道到什麼程度：看得懂 AI 寫在每一段上面的中文註解就夠了。語法完全不用背。

> **名詞**：Supabase 是一個開在網路上的資料庫服務，註冊就能用。
> 你需要知道到什麼程度：知道你的資料存在它那裡，權限規則也寫在它那裡。它內部怎麼實作可以不管。

這個案例要回答的問題

> 當網站開始有帳號、有資料、有很多人同時操作時，哪些事情不能只相信前端？

這個案例的主線是這一句：

> 前端的檢查是體驗，資料庫的規則才是保證。

目錄

開始之前

- 開始之前一定要看：這個案例只能用練習用的資料
- 學員路線圖
- 這個案例和案例一是什麼關係
- 這個案例做到最後，完成的是什麼？
- 這個專案是怎麼被建起來的
- 全課地圖
- 開始之前
  - 你需要準備的東西
  - 這個案例你要親手做什麼
  - 不需要裝 Docker
  - 環境檢查（五分鐘）
  - 認識 Claude Code Desktop 的畫面
  - 接上 GitHub（階段 17 才會用到）
- 這份手冊怎麼用

隨時可以翻回來查的

- 四張操作卡

    - 操作卡 1 和 AI 一起做事的七個步驟
    - 操作卡 2 哪些事 AI 可以自己做，哪些要先問你
    - 操作卡 3 一個能用的指令，有六個構件
    - 操作卡 4 卡住的時候怎麼辦
- 不會寫程式，怎麼審查 AI 的產出

階段

1. 先看成品，並確認哪一層壞了會怎樣
2. 判斷這個需求需不需要資料庫
3. 搞懂兩把金鑰，以及安全放在哪一層
    - Checkpoint 1（做完階段 3）
4. 建立專案，並搬進案例一的設計規範
5. 設計兩張資料表
6. 加上兩層權限：能不能碰這張表，和能碰哪幾列
7. 產生五筆示範窯次
8. 在自己電腦上驗證規則真的擋得住
    - Checkpoint 2（做完階段 8）
9. 把窯次資料接到畫面上
10. 做不用密碼的登入
11. 做預約與取消，並把錯誤翻成人話
12. 在正式環境上再驗一次規則

- Checkpoint 3（做完階段 12）

13. 重現問題：兩個人搶最後一個名額
14. 把「檢查」和「寫入」合成一個動作
15. 用同一個方法再驗一次
16. 堵住繞過網頁的那條路
17. 推上 GitHub 並部署成公開網址

做完之後

- 完成清單
- 離開這個範例，我會不會自己做？

- 常見卡關
- 名詞白話對照表
- 各階段預估時間

🗺️ 學員路線圖

起點	一個空資料夾、一個**新開的練習用** Supabase 專案、兩個測試用的信箱
過程	先判斷需不需要資料庫，再設計兩張表與兩層權限。用測試證明規則擋得住，才接畫面、登入與預約。最後親手重現一個漏洞並修掉
產出	一個可以登入、預約、取消的開窯預約網站，以及一份在正式環境驗過的權限規則
核心判斷	這個檢查該放前端還是資料庫：有人繞過網頁直接送請求，還擋不擋得住。階段 3 提出，階段 13 驗證

> **名詞**：**資料表**是資料庫裡的一張表，第一列是欄位名稱，下面一列一列是資料。這個專案有兩張：窯次表一列是一窯，預約表一列是一筆預約。
> 你需要知道到什麼程度：說得出這個網站需要哪幾張表、每張表有哪些欄位。資料實際怎麼存可以不管。

> **關於「階段」**：階段是這份手冊的章節單位。這個案例共 17 個（階段 1 → 階段 17），照順序做。

這個案例和案例一是什麼關係

同一個品牌網站的兩個部分。

	案例一做的	這個案例做的
是什麼	品牌首頁（一頁式）	開窯預約
用什麼做	參考成品是單一 HTML 檔；你自己做的版本是 Astro 專案	Astro + Supabase
為什麼不同	沒有資料、沒有互動，所以不需要資料庫	有帳號、有資料、有多人寫入，所以需要

正式上線時，它們會放在同一個網域下，例如 `vessel.tw` 和 `vessel.tw/kiln`。這個專案的導覽列第一項「品牌首頁」就是案例一的成品。它比案例一多兩個連回開窯場次的連結，其他完全相同。

> **名詞：Astro** 是一套做網站的工具。它讓你把共用的部分只寫一次。這個專案的導覽列和頁尾寫在 `Base.astro` 一個檔案裡，三個頁面都自動帶上。寫完之後，它把這些檔案合成一般的網頁，放上網路就打得開。
> 你需要知道到什麼程度：知道專案是用它建起來的，指令都由 Claude 執行。它的設定檔你不用碰。

一件要記住的事

案例一那份設計規範，搬到這個專案不用重寫。

同一組顏色、同一組字體、同樣的 2px 圓角、同一份禁用清單。階段 4 你會親手把它搬過來。

差別只有一處。**案例一沒有表單，這個案例有**，所以規範多了「五種狀態」那一節（輸入框、載入中、錯誤、成功、額滿）。

> 兩個網站的顏色或圓角對不起來，就表示 `CLAUDE.md` 的規範沒有搬過來，或自動檢查沒有生效。

這個案例做到最後，完成的是什麼？

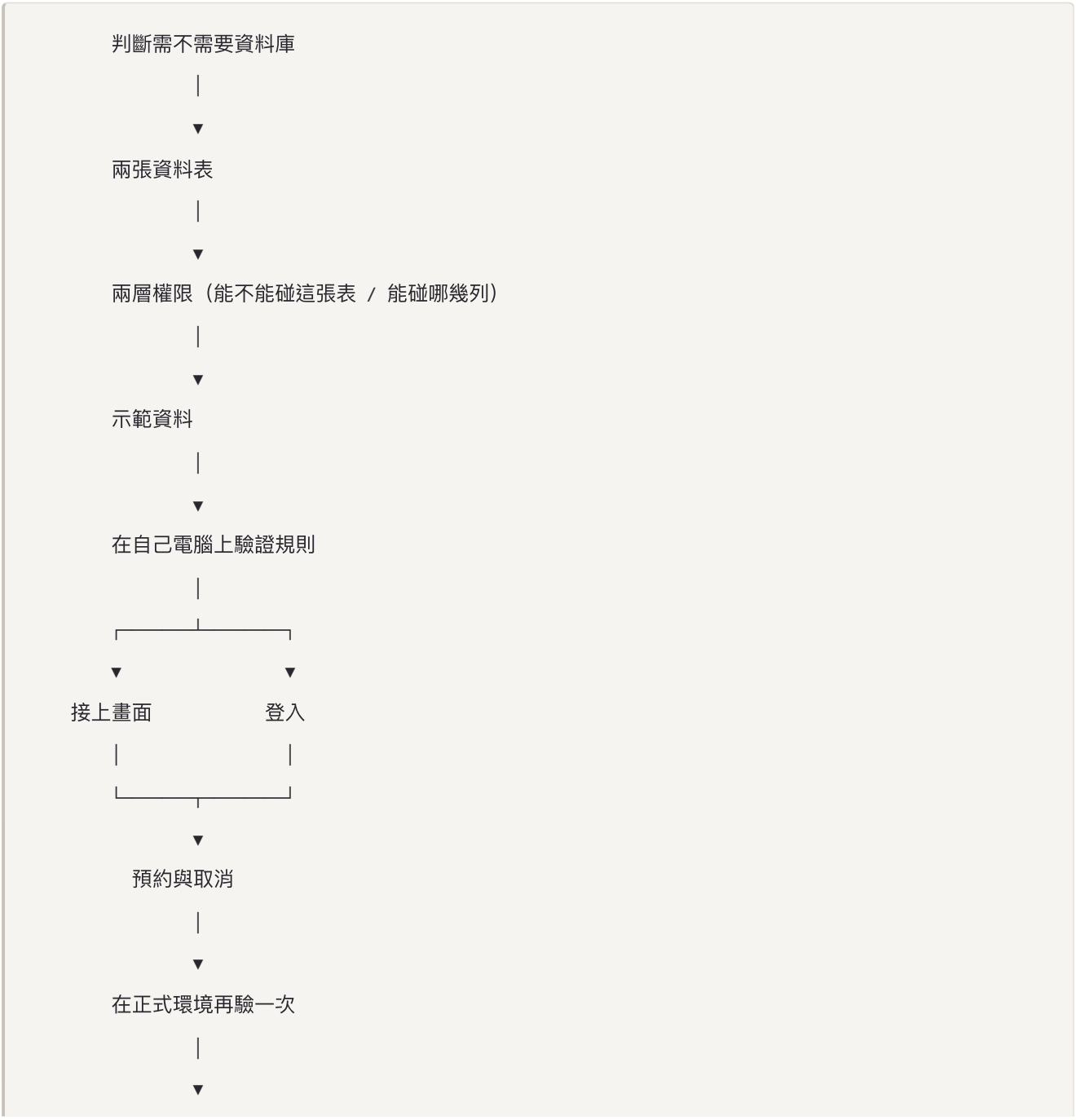
這個案例有兩個終點，中間隔著一個**故意留給你的漏洞**。

```
階段 12 結束  ▓▓▓
```

還缺什麼	為什麼這個案例不做
資料備份與監控	免費方案沒有，而且屬於維運範圍
濫用防護（限制同一個人短時間內大量請求）	值得做，但會讓這個案例變成兩倍長
免費方案閒置一週會自動暫停	這是方案限制，付費就沒有

這個專案是怎麼被建起來的

這個案例會先花一半的時間在「沒有畫面」的地方。



```
重現漏洞 → 修掉 → 再驗一次 → 堵住繞過
                  |
                  ▼
                部署
```

為什麼不能反過來

順序	如果反過來會怎樣
判斷 → 資料表	沒判斷就加資料庫，你會多出一個要備份、要付錢、要設權限、而且會掛掉的東西
資料表 → 權限	資料表決定「有什麼欄位」，權限決定「誰看得到」。表沒定案，權限規則寫了也要重寫
權限 → 示範資料	先有規則再放資料，你才能立刻測「這筆資料別人看不看得到」
示範資料 → 驗證	沒有資料就沒得測，測試會全部通過但什麼都沒證明
驗證 → 接畫面	規則沒驗過就接畫面，你會把「看起來正常」誤當成「安全」
接畫面 → 登入	先讓不用登入就看得到的部分會動，你才分得清楚哪些問題是登入造成的
登入 → 寫入	寫入需要知道「現在是誰」
寫入 → 正式環境驗證	本機的替代品和真的環境有差別，寫完才驗得完整
重現 → 修 → 再驗，再堵住繞過	先看到問題再修，你才會相信它真的存在。修完要用同一個方法再驗一次，再檢查有沒有另一條路寫得進去
全部 → 部署	**不要部署一個你已經知道有漏洞的東西**

全課地圖

階段	產出	重點
1	一個跑起來但沒有資料的網站	裝好套件，把成品跑起來看一次
2	一份四項條件的對照	拿四個條件逐條對照你的需求
3	AI 對四個「該相信前端還是資料庫」問題的回答	問 AI 四個問題，抄下兩句錯誤訊息
4	專案骨架 + 從案例一搬過來的設計規範與自動檢查	建 Astro 專案，搬案例一的規範
5	`schema.sql` 的兩張表	列出兩張表的欄位，要求中文註解

階段	產出	重點
6	兩層權限與三條政策	補上兩層權限與三條政策，執行兩次
7	`seed.sql` 五筆示範窯次	產生五筆假窯次並執行一次
8	`rls-test.sql` 七項會自己回報的測試	裝資料庫，換身分跑七項測試
9	看得到窯次與剩餘名額的首頁	填好 `.env`，把窯次列到首頁上
10	不用密碼的登入	設定白名單網址，做出寄信登入
11	預約與取消，含人話錯誤訊息	做預約與取消，把錯誤翻成人話
12	正式環境上七項全過的測試	造兩個真帳號，重跑七項測試
13	一次親手重現的超額預約	開兩個瀏覽器，同時搶最後一個名額
14	把檢查與寫入合成一個動作的資料庫函式	改成呼叫資料庫函式，不再先查再寫
15	同一個實驗，這次只有一筆成功	用階段 13 一樣的方法再做一次
16	收回直接寫入的權限 + 改過的測試	先重現繞過，再收回權限，重跑七項測試
17	一道上線關卡 + 一個公開網址	裝上線關卡，推上 GitHub，設環境變數後部署

開始之前

你需要準備的東西

項目	說明
Claude Code Desktop	桌面版應用程式，到 claude.com/download 下載安裝，用你的 Claude 帳號登入。整個案例都在裡面完成
一個付費的 Claude 方案	**免費方案打不開 Code 分頁。** 需要 Pro、Max、Team 或 Enterprise 其中一種。上課前先確認
Windows 使用者多一步	第一次打開 Code 分頁之前要先裝 Git for Windows，裝完把應用程式關掉重開。macOS 不用
Node.js 22.12 以上	下面〈環境檢查〉會確認，沒有的話照那裡的方法裝
一個 Supabase 免費帳號	到 supabase.com 註冊。**建議先做，註冊要幾分鐘**
兩個可以收信的 email	階段 12 要用兩個不同的使用者測試。Gmail 的別名就可以（見階段 12）
一個 GitHub 帳號	階段 17 部署要用，Vercel 也用它登入。先去 github.com 註冊好，再照下面〈接上 GitHub〉登入一次
品牌首頁的成品檔	階段 4 要用。就是課程附件裡的 `_reference-vessel.html`，不用另外準備
課程附件	到課程頁面下載 `case-03-kiln.zip`，解壓縮到桌面。階段 1 會用

這個案例你要親手做什麼

動作	難度
複製手冊上的 Prompt，貼給 Claude	不難，這是主要工作
在 Supabase 的網站上點設定	不難，每一步都有說明
把 SQL 整份複製、貼到 Supabase 網站上按 Run	不難，你不用看懂它
決定要不要允許 Claude 執行某個動作	不難，它會跳卡片問你
判斷 AI 寫的規則對不對	**這是這個案例的重點**
自己打指令	不用。指令都由 Claude 執行

動作	難度
看懂 SQL 或 JavaScript	不用。你只需要讀得懂中文註解

不需要裝 Docker

你可能聽過「用資料庫要先裝 Docker」。那要看你怎麼用：

用法	需要什麼	這個案例
Supabase 雲端	什麼都不用裝，開個網頁註冊就好	用這個
在自己電腦跑一整套 Supabase	需要 Docker	不用

> **但這不代表本機測試不重要。** 階段 8 會用一個**不需要 Docker** 的方式，在你自己的電腦上驗證規則。改一次規則，三十秒就能驗完一次。

環境檢查（五分鐘）

先做完這一步再開始。 環境沒弄好就往下走，後面每一步都會出錯，而且錯誤訊息看起來跟環境無關。

動手

打開 Claude Code Desktop，按左上角的 **Code** 分頁，開一個新對話。**Project folder** 隨便選一個空資料夾就好。它在輸入框上方那一排，位置見下一節的定位圖。把這段貼進去：

貼給 AI

你只需要修改：沒有。整段照貼，四個問題一個都不要刪。

```
幫我檢查這台電腦能不能做網站開發。請逐項回報，先不要幫我安裝任何東西：

1. Node.js 有沒有裝？版本是多少？（我需要 22.12 以上）
2. npm 能不能正常執行？
3. 我對目前這個資料夾有沒有寫入權限？
4. 能不能連到 npm 的伺服器？（隨便查一個套件的版本就好）

每一項都用「可以」或「不行」開頭。
不行的話告訴我：問題是什麼、對一個沒有程式背景的人來說最簡單的解法是什麼。
```

四項都「可以」就準備好了。

常見狀況

訊息	意思	怎麼辦
`command not found: npm`	這台電腦沒有裝 Node.js	到 nodejs.org 下載 **LTS** 版本（長期支援版，比較穩），一路按下一步
版本是 18、20 這種數字	太舊了	同上，重裝一次 LTS 版本
Windows 上 Code 分頁打不開	少了 Git for Windows	到 git-scm.com 安裝，裝完把應用程式重開
`EACCES` 或 `permission denied`	那個資料夾不讓你寫東西	換一個資料夾，例如在桌面上新建一個
安裝一直卡住不動	網路擋住了 npm	換一個網路（手機熱點通常可以）。公司電腦的話問一下 IT
Views 選單裡沒有 Browser 面板	應用程式版本太舊	選單列的 **Claude → Check for Updates**（Windows 是 **Help → Check for Updates**）
硬碟空間不足	一個專案大約要 200–300MB	清一些空間再來

認識 Claude Code Desktop 的畫面

打開應用程式之後，**視窗左上角有 Home 和 Code 兩個分頁。這個案例只會用到 Code 分頁。**

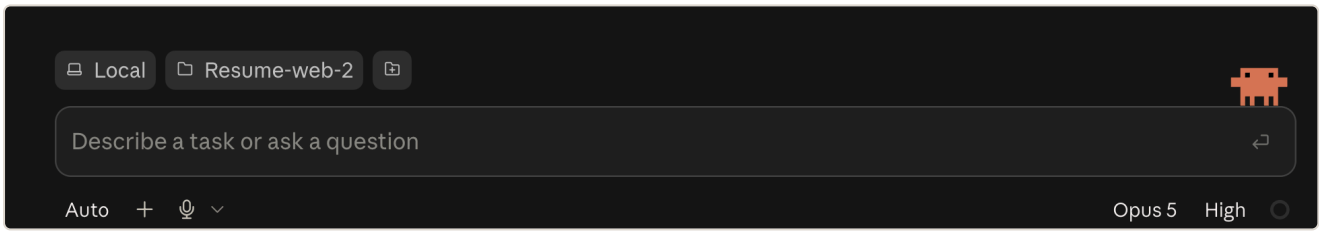
（`Chat` 和 `Cowork` 不是分頁。它們是 Home 畫面輸入框右下角的兩個切換鈕，這個案例不會用到。）

🖥️ 介面定位圖：主要會碰到六個地方

主要會用到下面六個位置，找不到某一項時回來看這張表。Views 選單裡還有一個 Terminal，只有手冊寫「你自己打」的指令才會用到。



> **你要看的是什麼**：① 在**視窗最左上角**，和 Home 並排。打開應用程式時會停在 **Home**，**每次都要先按一下 Code**，這份手冊講的所有東西才會出現在你眼前。



> **你要看的是什麼**：② ③ ④ 三個設定全部集中在**視窗最下方這一條**，不在任何選單裡。左上角的 `Local` 是 Environment。旁邊的資料夾名稱（圖上是 `Resume-web-2`）是 Project folder，再旁邊那個小按鈕可以換資料夾。輸入框**下面**那一排的最左邊（圖上是 `Auto`）就是權限模式選單。大部分時間要把它切成 **Plan** 或 **Manual**。右下角的 `Opus 5` 是模型，用預設的就好。

#	名稱	在哪裡找	用途
①	**Code**（分頁）	視窗**左上角**，和 Home 並排（見上面第一張圖）	整個案例只待在這個分頁。開啟時停在 Home，要先按 Code
②	**Environment**	輸入框**上方**那一排的第一個（上圖是 `Local`）	選 **Local**，檔案才會建在你自己的電腦上
③	**Project folder**	緊接在 Environment 右邊（上圖是 `Resume-web-2`）	指定 Claude 只能在哪個資料夾裡工作
④	**模式選單**（Plan / Manual …）	輸入框**下方**那一排的最左邊（上圖是 `Auto`），隨時可切	手冊寫「切到 Plan」的地方先用 **Plan** 看計畫，確認後切 **Manual**。其餘直接用 **Manual**

#	名稱	在哪裡找	用途
⑤	**Views → Browser**	選單列的 **Views**（上圖沒拍到）	直接預覽你做的網站，可以拖到對話框旁邊並排
⑥	**Views → Diff**	同一個 Views 選單（上圖沒拍到）	看 Claude 這次改了哪幾行。**這是你最主要的驗收工具**

開始一個對話之前，先設定三件事

位置都見上面的定位圖，這裡只講要選什麼：

1. **Environment** 選 `Local`
2. **Project folder** 選那一階段手冊指定的資料夾
3. **模式選單** 見下一段

模型用預設的就好。

權限模式：決定 AI 什麼時候要先問你

模式選單在輸入框**下方**那一排的最左邊（見上面的介面定位圖），隨時可以切換。

模式	白話	用在哪
Plan	只規劃，不動手	手冊寫「切到 Plan」的地方用它，先看 Claude 打算怎麼做
Manual	每一個改動都先問你	確認計畫之後執行用這個。其餘時間都用它
Accept edits	自動接受改檔，但執行指令還是會問	熟了之後可以換成這個，比較快
Auto	自動執行，背後有安全檢查	這個案例用不到
Bypass permissions	什麼都不問	不要用

> **Plan 模式只套用在目前這個對話**，開新對話要重選。其他模式會記住你在這個資料夾的選擇。

> 舊版本的介面把這幾個模式叫做 Ask permissions、Auto accept edits、Plan mode。名字不同，意思一樣。

三個好用的面板

從 **Views** 選單打開，可以拖到旁邊並排著看：

- **Diff**：Claude 這次改了哪幾行，一行一行對照。**這是你最主要的驗收工具**
- **Browser**：直接預覽你做的網站，不用另外開瀏覽器
- **Terminal**：自己打指令的地方。手冊寫「你自己打」的指令才會用到

Claude 動手之前會問你

在 Manual 模式下，Claude 每次要改檔案或執行指令都會停下來。它會跳出一張卡片等你決定。

看不懂那張卡片在說什麼的時候，不要按允許。 直接在對話框問它：「你剛剛那個動作是要做什麼？為什麼需要？」

接上 GitHub（階段 17 才會用到）

階段 17 要讓 Claude 建一個 GitHub 專案，再把程式碼推上去。它用的是 GitHub 官方的指令列工具 `gh`。這一節把它裝好、登入一次。之後不用再做。

> **名詞**：`gh` 是 GitHub 官方的指令列工具。Claude 用它建專案、推程式碼，不用開網頁。
> 你需要知道到什麼程度：知道它要先登入一次。登入在瀏覽器裡完成，密碼不會經過 Claude。

📋 貼給 AI（裝 gh 並登入）

貼給 AI

你只需要修改：沒有。整段照貼。**第 1 點「我同意之後再裝」不要刪。**

```
幫我安裝 GitHub 的指令列工具 gh，然後登入我的 GitHub 帳號。

1. 先檢查 gh 是不是已經裝了。沒有的話，告訴我你要用什麼方式安裝、
   要執行什麼指令，我同意之後再裝
2. 登入用瀏覽器的方式：gh auth login --web --git-protocol https --hostname github.com
   它會印出一組一次性代碼和一個網址，請把這兩個原樣給我看
3. 我在瀏覽器完成之後，用 gh auth status 確認登入成功
```

你自己做（在瀏覽器完成登入）

1. 打開 Claude 給你的網址（`https://github.com/login/device`）
2. 用你的 GitHub 帳號登入，貼上那組代碼，照畫面指示授權
3. 回到 Claude Code，跟它說「好了」

> **它要你輸入密碼或 token 怎麼辦**：不要在對話框裡輸入。回它「請改用瀏覽器登入的方式」。

✅ 怎麼知道做對了

- [ ] `gh auth status` 顯示已經登入 github.com，後面是你的帳號名稱

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
Mac 上 Claude 說沒有 Homebrew	這台電腦沒有 Homebrew 這個安裝工具	到 cli.github.com 下載安裝檔自己裝，裝完回來從第 2 點繼續
瀏覽器說代碼無效	代碼過期了，或用了上一次的代碼	「代碼過期了，請重新執行一次登入」
Windows 或公司電腦裝不了 `gh`	安裝可能需要 IT 或系統管理員權限	「請說明用 winget 安裝 gh 的方式；如果裝不了，教我改用 GitHub 網站手動建立專案」

這一步失敗可以跳過嗎：可以，只影響階段 17。

四張操作卡

這四張卡不是這個案例的內容，是和 AI 一起做事的通用方法。 每個案例裡都一樣，你之後做自己的專案也用得上。建議印出來，或者截圖存在手機裡。

不用現在背下來。先掃過一遍知道有這四張，遇到下面這些情況再翻回來：

情況	操作卡
要開始一件新的工作，不知道從哪一步下手	**操作卡 1**：七個步驟的順序
Claude 說它要做某件事，你不確定該不該讓它做	**操作卡 2**：哪些可以放手、哪些要你先點頭
想不出指令怎麼寫，或者寫了但它答非所問	**操作卡 3**：一個能用的指令有哪六塊
卡住超過 20 分鐘	**操作卡 4**：卡住的三步處理法
做到後面，Claude 開始違反前面講好的規則	**操作卡 4**：最後一節〈對話太長的時候怎麼辦〉

手冊各階段的 🆘 只處理那一步特有的問題。**手冊沒寫到的情況，回來看這四張。**

操作卡 1｜和 AI 一起做事的七個步驟

```
Step 1  先把需求講清楚
        要做什麼 / 不做什麼 / 有什麼限制 / 怎麼算完成

Step 2  新的、大的改動，先讓 AI 規劃，不要直接改
        什麼時候切到 Plan：
          要做新功能、要大範圍修改、規格還沒談定
        切到 Plan 模式，並明講：
        「先不要修改任何檔案。請告訴我你準備怎麼做、
          會改哪些檔案、每一步的目的，以及可能的風險。」
        手冊寫「這一步用 Manual」的階段，
        是照已經談定的規格或計畫做，直接跳到 Step 4

Step 3  看過計畫，再切回 Manual 讓它執行
        看三件事：有沒有誤解需求 / 有沒有多做不需要的事 /
        有沒有要碰不該碰的檔案或外部資源
        確認完一定要切回 Manual，Plan 模式下它不會動任何檔案
```

```
Step 4  一次只做一小段
        執行 → 看結果 → 驗收 → 確認 → 再往下一步
        不要一次叫 AI 重做整個專案

Step 5  照手冊的驗收條件檢查
        不要用「好像可以」「看起來差不多」
        要用看得到的結果、量得到的數字、跑得出來的測試
        不知道該看哪裡、該怎麼量，就把這一句貼過去：
        「請列出這次改動裡，我在畫面上看不出來、
          但會出錯的地方，以及我要怎麼試。」

Step 6  失敗時不要猜
        把錯誤訊息整段貼給 Claude，再加一句：
        「請先不要修改任何東西。先告訴我最可能的三個原因，
          以及你準備先檢查哪一個。」

Step 7  修完之後重新驗收一次
        「沒有錯誤訊息」不等於「做對了」
```

操作卡 2｜哪些事 AI 可以自己做，哪些要先問你

層級	內容
AI 可以自己做	讀取專案檔案 / 修改本機檔案 / 執行 build（打包） / 執行測試 / 執行檢查 / 分析錯誤 / 建立本機的存檔紀錄（commit）
執行前要你確認	安裝新的工具 / 改動電腦的全域設定 / 大量刪除檔案 / 存取專案資料夾以外的地方
一定要你點頭，它才能做	建立 GitHub 專案 / 第一次把程式碼推上去 / 建立雲端資源 / 部署到公開網址 / 修改正式資料 / 付費 / 發送訊息 / 刪除遠端資源

第三層的原則是：**對外的、要花錢的、不可逆的，AI 不能自己決定。**

重點不是「一定要你親手按」，而是**它動手之前你必須先看到它要做什麼**。這幾件事做下去就收不回來，你要有機會在執行前喊停。正確的說法是：

> 請你先準備好需要的內容與指令，把即將進行的對外操作說明給我聽，我確認之後你再執行。

操作卡 3｜一個能用的指令，有六個構件

手冊上的每一個 Prompt 都是照這六個構件組出來的。**要寫自己的指令時，照同一個順序填。**

#	構件	寫什麼
1	要做什麼	一句話講完
2	不要做什麼	負面清單。這一項最常被漏掉，也最有效
3	限制是什麼	技術、數值、要遵守哪份文件
4	先規劃還是先動手	「先不要改任何檔案，先告訴我你打算怎麼做」
5	怎麼算完成	看得到、量得到、跑得出來的條件
6	哪裡要停下來問我	對外的、要花錢的、要裝東西的

手冊上的 Prompt 怎麼改成你自己的

- `〈…〉` 裡的內容**一定要換掉**
- 其餘的可以照你的專案改寫，但**六個構件盡量都留著**
- **不確定哪裡能動**：手冊上每一段可以貼的 Prompt，上面都有一行「**你只需要修改**：…」。照那一行做就好，沒被點到的部分整段照貼，不用看懂

三個馬上可以用的句型

貼給 AI

```
先不要寫程式碼。用沒有程式背景的人聽得懂的方式，
告訴我你打算怎麼做、分成哪幾步、每一步在解決什麼問題。
我確認之後你再寫。
```

貼給 AI

```
只做 〈這一段〉，其他的等一下再說。
```

貼給 AI

有錯先告訴我是什麼，不要直接改。

操作卡 4｜卡住的時候怎麼辦

依序做這三步

第 1 步　照那個階段的「🆘 如果沒有成功」做一次

第 2 步　把下面的 Recovery Prompt 貼給 Claude，一次只處理一個原因

第 3 步　還是不行 → 先跳過

每個階段的 🆘 都會註明「這一步失敗可以跳過嗎」

卡超過 20 分鐘還沒解決，繼續卡下去沒有意義

Recovery Prompt

貼給 AI

我照著教材做到這一步，但沒有得到預期的結果。

我沒有程式背景。

教材說我應該看到：

〈貼上手冊裡那一階段的驗收條件〉

我實際看到的是：

〈描述畫面，或整段貼上錯誤訊息〉

請先不要修改任何東西。先告訴我：

1. 最可能是哪三個原因
2. 你準備先檢查哪一個

3. 為什麼是這一個
4. 你需要我提供什麼資訊

一次只處理一個原因。

「整段貼上錯誤訊息」不要偷懶。 真正有用的線索，常常不在最紅、最顯眼的那一行。它在上面幾行不起眼的路徑或版本號裡。

「先不要修改任何東西」也是刻意的。 讓 AI 自己想辦法繞過去，它可能會裝一堆你不知道的東西，或是改掉某個設定。問題會藏得更深。先知道原因，再讓它改。

對話太長的時候怎麼辦

症狀：做到後面，Claude 開始違反前面講好的規則，或是重做已經做過的事。對話越長，前面講過的內容在它眼裡佔的份量越低。

情況	做法
同一件工作還沒做完，對話已經很長	在對話框輸入 `/compact`。它會把前面的對話壓縮成摘要，接著做
這一件做完了，要開始下一件	先確認談好的規則都寫進 `CLAUDE.md`，再輸入 `/clear` 或開一個新對話

`/clear` 之後，Claude 只看得到 `CLAUDE.md` 和專案裡的檔案。談好但沒寫進檔案的決定，換了對話就不見了。接著上一段的 Prompt，例如「照你剛剛列的做」，要和前一段在同一個對話裡做。中間不要 `/clear`。換對話之前先問一句：

貼給 AI

我要開一個新對話繼續做。

這個對話裡談定、但還沒寫進 CLAUDE.md 或任何檔案的決定，請列出來。

我確認之後，請把需要保留的寫進 CLAUDE.md。

不會寫程式，怎麼審查 AI 的產出

下面七個檢查，每一個都不需要看懂程式碼。第五個是這個案例特別加的，因為它處理的是別人的資料。

一、先要它用白話解釋，再讓它寫

在任何一個 Prompt 的最後加上這一段：

貼給 AI

> 先不要寫程式碼。用沒有程式背景的人聽得懂的方式，
> 告訴我你打算怎麼做、分成哪幾步、每一步在解決什麼問題。
> 我確認之後你再寫。

二、要求中文註解，然後只讀註解

貼給 AI

> 請在每一段程式碼上面用中文註解說明它在做什麼，
> 以及「為什麼要這樣做」。

然後你只讀註解。 註解看起來合理、和你的需求對得上，多半就沒問題。註解含糊（「處理資料」「初始化」）或和你要的不一樣，**停下來問**。

這個案例的 SQL 全部都會有中文註解，那是你唯一需要讀的東西。

三、讓它寫會自己回報的測試

階段 8 就是在做這件事。

貼給 AI

> 請寫一份測試，每一項都自己印出 ✓ 或 ✗，
> 讓我執行一次就知道有沒有過，不需要看懂程式碼。

你不用讀懂 SQL。 看那七行是 ✓ 還是 ✗，就知道規則有沒有生效。

四、問它自己最可能錯在哪

貼給 AI

這段程式碼如果會出問題，最可能是哪裡？

請列出三個，並告訴我各自的症狀長什麼樣。

五、反過來問（這個案例特別重要）

貼給 AI

如果我是壞人，想偷看別人的預約資料，我會從哪裡下手？

這個專案目前擋得住嗎？請逐項說明。

這一問常常會逼出它自己剛剛留下的漏洞。

六、確認它只改了該改的東西

貼給 AI

你這次改了哪些檔案？每一個各改了什麼、為什麼？

七、紅旗清單：看到這些就停下來問

看到這個	為什麼要警覺
`service_role`	這把金鑰無視所有規則。**它不該出現在前端專案的任何地方**
`disable row level security`	有人把權限規則整個關掉了
`using (true)` 出現在**預約**相關的規則上	意思是「誰都可以」。用在公開資料沒問題，用在私人資料就是外洩
密碼、金鑰被寫死在程式碼裡	這些應該放在設定檔，而且設定檔不進版本控制

看到這個	為什麼要警覺
`.env` 出現在要上傳的檔案清單裡	設定檔不該進版本控制
錯誤訊息直接把資料庫的原文丟到畫面上	會洩漏內部結構，而且使用者看不懂

看到上面任何一項，就把那一行貼給 Claude 問：

貼給 AI

這一行是做什麼的？有沒有安全問題？如果有人惡意利用，會發生什麼事？

這份手冊怎麼用

每個階段最多分成三塊：

```
┌─ ⚖️ 動手前先想（有些階段才有）──────
│  這一步要做的判斷，以及判斷的依據
└──────────────────────────────────

┌─ 🛠️ 動手做 ──────────────────────
│  🧰 這一步的產出
│  📋 貼給 AI
│  ✅ 怎麼知道做對了
│  🆘 如果沒有成功
└──────────────────────────────────

┌─ 📖 為什麼這樣做 ────────────────
│  🎯 這一步的重點
│  🧭 這一步在做什麼
│  🧩 它在建構流程的位置
│  💡 那段指令為什麼那樣寫
└──────────────────────────────────
```

先想判斷，再動手，原理在後。 讀完階段開頭那三行提示和〈⚖️ 動手前先想〉就可以動手，做完再回頭讀為什麼。

階段 2 是唯一的例外。它的產出是一個判斷，所以那一階段先讀後做。

這個案例有一半的階段沒有畫面變化。階段 5 到 8 只會改 `supabase/` 裡的檔案，網站畫面不會變。每個階段的 🧰 都會告訴你這一步做完畫面長什麼樣，包含「畫面沒有變化」這種答案。

Prompt 裡的 `〈…〉` 是要換成你自己內容的地方。

階段 1　先看成品，並確認哪一層壞了會怎樣

📖 讀完約 5 分鐘　🛠️ 動手約 15 分鐘

- **這一步一定要懂**：畫面說「尚未連上資料庫」是**正確的結果**，成品資料夾裡沒有附金鑰。前端壞了是版面跑掉，資料庫壞了是版面正常但沒有資料。
- **這一步可以先不懂**：安裝的時候跑了哪些指令。這一步不寫任何程式碼。
- **做完後你要能用自己的話回答**：資料庫壞掉為什麼比前端壞掉危險？

🛠️ 動手做

🧰 這一步的產出

- 在 Browser 面板裡跑起來的成品
- 一個顯示「尚未連上資料庫」的畫面（**這是正確的結果**）

📋 貼給 AI

1. 到課程頁面下載 `case-03-kiln.zip`，解壓縮到桌面
2. 開一個新對話，**Project folder** 選那個資料夾
3. 貼上：

貼給 AI

你只需要修改：沒有。整句照貼。

> 幫我把這個專案需要的東西裝好，然後跑起來。

4. 從 **Views** 選單打開 **Browser** 面板

> 這個資料夾是**看目標用的**。從階段 4 開始，你會在一個全新的空資料夾裡重做一次。

✅ 怎麼知道做對了

- [ ] 網站在 Browser 面板裡打得開
- [ ] 畫面上有導覽列和頁尾，排版正常

- [ ] 往下捲到「近期窯次」那一段，畫面說「尚未連上資料庫」，**而且它有告訴我要做什麼**
- [ ] 我說得出「前端壞了」和「資料庫壞了」在畫面上的差別



> **你要看的是什麼**：近期窯次那一段寫著「尚未連上資料庫」，下面說明要做什麼。這句話不在畫面最上面，要往下捲才看得到。

第三項特別重要。錯誤畫面要說人話，這是這個案例的設計要求之一。階段 9 你會被要求做出同樣的東西。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
一片白，還有一堆紅色文字	這不是預期的畫面	整段錯誤訊息貼給 Claude，再加一句：「請先不要修改任何東西。先告訴我最可能的三個原因，以及你準備先檢查哪一個。」
安裝跑很久	第一次安裝要下載很多東西	等兩分鐘。超過五分鐘就換個網路
`command not found: npm`	沒裝 Node.js	到 nodejs.org 下載 LTS 版本

這一步失敗可以跳過嗎：可以。但建議想辦法看到那個「尚未連上資料庫」的畫面，它是階段 9 的驗收標準。

📖 為什麼這樣做

🎯 這一步的重點

看得出「前端」和「資料庫」是兩層不同的東西，而且它們壞掉的方式完全不一樣。

🧭 這一步在做什麼

先把成品跑起來看一次。

但這一次你看到的畫面會說「尚未連上資料庫」，而且那是刻意的。

原因是成品資料夾裡沒有附上金鑰。那把金鑰開的是參考成品的 Supabase 專案，不會附給學員。所以你會看到一個很重要的畫面：

> 這個網站在沒有資料庫的時候仍然打得開，只是沒有內容。

版面、字體、排版是一層，這一層叫前端。資料是另一層，這一層是資料庫。這兩層在這個案例會一直分開講，因為**它們壞掉的方式完全不一樣**：

哪一層壞了	你會看到什麼
前端	版面跑掉、字體不對、按鈕位置怪
資料庫	版面完全正常，但沒有資料，或者資料是錯的

第二種比第一種危險得多，因為它看起來很正常。這個案例要教的就是怎麼抓第二種。

🧩 它在建構流程的位置

流程開始之前。

💡 那段指令為什麼那樣寫

只有一句「幫我把這個專案需要的東西裝好，然後跑起來」。第一次安裝會跑一兩分鐘，那是正常的。

階段 2　判斷這個需求需不需要資料庫

📖 讀完約 6 分鐘　🛠️ 動手約 10 分鐘

> - **這一步一定要懂**：那四個條件是多人寫入、不用改程式就會變、需要查詢、有權限之分。一項都不中，就不需要資料庫。
> - **這一步可以先不懂**：資料庫實際怎麼運作、Supabase 有哪些功能。這一步不寫任何程式碼。
> - **做完後你要能用自己的話回答**：一個只展示十張照片的攝影網站，為什麼不需要資料庫？

📖 先讀懂

🎯 這一步的重點

判斷一個需求需不需要資料庫。

🧭 這一步在做什麼

「加一個資料庫」是最容易被當成理所當然的決定之一。

但資料庫不是免費的。加了之後，你多一個要備份、要付錢、要設權限、而且會掛掉的東西。用不到的話，那些全都是白付的成本。

🧩 它在建構流程的位置

整條流程的第一個關卡。這一關沒過，後面十五個階段都不該做。

⚖️ 這一步的判斷

這個需求需不需要資料庫？

先問自己一個問題：

> 把資料直接寫在程式碼裡（寫死），什麼東西會壞掉？
>
> **答不出來，就不需要資料庫。**

真正需要資料庫的只有四種情況。逐項對照你的需求：

條件	白話	開窯預約符合嗎
多人寫入	別人寫的東西，我要看得到	✅ 別人預約了，我看到名額變少

條件	白話	開窯預約符合嗎
不用改程式就會變	資料自己會變，不是等工程師改	✅ 隨時有人預約、有人取消
需要查詢	要篩選、排序、統計，不只是列出來	✅ 只列未來的場次、算剩餘名額
有權限之分	誰能看什麼、誰能改什麼不一樣	✅ 只能看自己的預約

四項全中，所以開窯預約需要資料庫。

反過來看一個不需要的例子。 一個只展示十張城市照片的攝影網站。那十個場景是作者自己策展好的內容，一個人寫、上線一次就好。沒有人會寫入，也沒有權限之分。四項一項都不中，所以它不需要資料庫。那十筆資料寫在程式碼裡是正確的決定。

多加一個資料庫的代價

- 多一個會掛掉的東西（外部服務、網路、免費額度）
- 多一個要設權限的地方，**權限寫錯就是資料外洩**
- 多一個要備份、要付錢的東西
- 每次改資料表，正式環境要跟著改一次

只有你自己用的東西，多一個資料庫影響不大。要維護三年的專案，每改一次資料表，正式環境就要跟著改一次。

💡 這一步的指令要包含什麼

這一步不需要下指令，你只要對照那四個條件。

但如果你想拿自己的專案練習，可以問：

貼給 AI

你只需要修改：只有 `（你的需求）`。四個條件與最後兩句照貼。

```
我想做 （你的需求）。

請幫我判斷它需不需要資料庫。判斷依據是四個條件：
多人寫入、不用改程式就會變、需要查詢、有權限之分。

請逐條回答符不符合，並說明理由。
最後給我一個明確的建議，以及如果不用資料庫的替代做法。
```

🛠️ 動手做

🧰 這一步的產出

不會多出任何檔案。你會得到一個判斷。**把那四個條件抄在筆記上**，換一個專案還是拿它們對照。

✅ 怎麼知道做對了

- [ ] 我說得出那四個條件
- [ ] 我舉得出一個「看起來需要但其實不需要」的例子。常見答案：個人部落格、作品集、公司簡介頁。
- [ ] 我解釋得出為什麼那個攝影網站不需要資料庫

🆘 如果沒有成功

你可能會遇到：自己的專案四項只中一兩項，不確定該不該做。

判斷方法：只中「需要查詢」一項的話，通常不需要資料庫。用一個資料檔加上前端篩選就夠了。**只要中了「多人寫入」，幾乎一定需要資料庫。**

這一步失敗可以跳過嗎：不行，但它只是一個判斷，不會卡住。

階段 3　搞懂兩把金鑰，以及安全放在哪一層

📖 讀完約 10 分鐘　🛠️ 動手約 15 分鐘

- **這一步一定要懂**：那把會被打包進網頁的金鑰**本來就是公開的**。擋住別人的是資料庫裡的規則。
- **這一步可以先不懂**：PostgREST、GoTrue、通行證（JWT）各自怎麼運作。這一步不寫任何程式碼。
- **做完後你要能用自己的話回答**：「只能看到自己的預約」是靠哪一層在擋？

名詞：**打包**是網站上線前的一道手續，把你寫的檔案組裝成瀏覽器看得懂的成品。
你需要知道到什麼程度：知道**這道手續會把設定值直接寫死進檔案裡**。所以金鑰藏不住，也所以改了設定要重跑一次才算數（階段 17 會親身遇到）。

⚖️ 動手前先想

這個檢查放前端還是資料庫？ 這一題會一路問到階段 16。

下面那段 Prompt 的第 2 題，就是這一題的第一個版本：「如果我在前端寫『額滿就把預約按鈕藏起來』，這能不能保證不會超額？」答案在〈📖 為什麼這樣做〉，貼完 Prompt、看完 Claude 的回答再去對照。

🛠️ 動手做

🧰 這一步的產出

不會多出檔案。你會得到 AI 對四個問題的回答，以及兩句要抄下來的錯誤訊息。

📋 貼給 AI

把模式切到 **Plan**（只規劃、不動手），然後：

貼之前先自己回答第 2 題：額滿就把預約按鈕藏起來，能不能保證不會超額？答對答錯都好，**記住自己的答案**，階段 13 會回來問你。

貼給 AI

你只需要修改：只有第一段的需求描述（如果你做的是別的題材）。**下面四個問題與最後那一句照貼**。它對這四個問題的回答，就是這一步的產出。

我要做一個開窯預約的網站：任何人都能看場次和剩餘名額，
登入之後可以預約，而且只能看到自己的預約。
前端是純靜態網站，用 Supabase 當資料庫，
可以公開的那一把金鑰會被打包進前端。

先不要寫任何程式碼。請用「沒有程式背景的人也聽得懂」的方式回答四個問題：

1. 既然那把金鑰是公開的，那「只能看到自己的預約」這件事，
   是靠什麼保證的？請具體說出是哪一層在擋。
2. 如果我在前端寫「額滿就把預約按鈕藏起來」，
   這能不能保證不會超額？為什麼？
3. 「能不能碰這張表」和「能碰這張表的哪幾列」是兩件不同的事嗎？
   請各舉一個實際會看到的錯誤訊息。
4. 這個需求裡，有哪些檢查「前端做了也沒用」，
   有哪些檢查「前端該做，但只是為了體驗」？

請用沒有程式背景的人聽得懂的方式說明，不要用程式碼。

兩句要抄下來的錯誤訊息

第 3 題的答案應該包含這兩句。**把它們抄在手邊**，之後每次畫面壞掉都會用到：

你看到	代表哪一層擋下來	白話
`permission denied for table ...`	第一層	你連這張表都碰不到
`new row violates row-level security policy`	第二層	你碰得到這張表，但你想寫的那一列不符合規則

✅ 怎麼知道做對了

- [ ] 我說得出兩把鑰匙的差別，以及哪一把絕不能外流（答不出來就回頭看 AI 對第 1 題的回答）

- [ ] 我先回答了第 2 題（不管答對答錯），而且我記得自己的答案
- [ ] 我抄下了那兩句錯誤訊息
- [ ] 我可以用自己的話說出「能不能碰這張表」和「能碰哪幾列」的差別

第二項聽起來很小，但**階段 13 會回來問你**。

🆘 如果沒有成功

你可能會看到：AI 的回答都在講「金鑰要保管好」。

怎麼辦：

貼給 AI

> 你的回答一直在講金鑰要保管好，但我問的不是這個。
>
> 那把金鑰本來就是公開的，使用者按 `F12` 就看得到。
> 在這個前提下，「只能看到自己的預約」是靠什麼保證的？
>
> 請具體說出是哪一層、哪一個機制在擋。

這一步失敗可以跳過嗎：不行。這是這個案例的觀念基礎。

📖 為什麼這樣做

🎯 這一步的重點

理解三件事：Supabase 給了你什麼、兩把金鑰的差別、以及**安全放在哪一層**。

🧭 這一步在做什麼

以前要做一個有帳號、有資料的網站，你需要三樣東西。一台伺服器、一個資料庫、一套自己寫的後端程式。

Supabase 把後兩樣打包好，而且讓瀏覽器可以直接跟它講話。所以這個案例**完全不用寫後端**，也不用租伺服器。

🧩 它在建構流程的位置

判斷之後、動工之前。這一步建立的觀念會貫穿之後每一個階段。

一、Supabase 給了什麼

它是一個資料庫，外面加了幾層讓瀏覽器可以直接用的東西：

你拿到的	它其實是什麼	白話
資料庫	PostgreSQL	業界最主流的資料庫之一，不是它自己發明的
自動介面	PostgREST	你建好資料表，它自動生出「網路上怎麼拿到這些資料」的窗口
登入	GoTrue	處理註冊、登入、發通行證
權限	PostgreSQL 內建的功能	也不是它發明的

換一家資料庫服務，這個案例的觀念還是同一套。你要記的不是 Supabase 的選單位置。

二、兩把金鑰

Supabase 會給你兩把鑰匙：

鑰匙	放哪裡	權限
可以公開的那一把 新制叫 Publishable，舊制叫 anon	**前端，公開**	完全由你設定的規則決定
不能外流的那一把 新制叫 Secret，舊制叫 service_role	**絕不外流**	**無視所有規則，什麼都能做**

> Supabase 正在把舊名字換成新的，舊制之後會淘汰，時間以 Supabase 官方公告為準。 **兩套目前都能用，觀念完全一樣。**

可以公開的那一把金鑰會被打包進網頁，使用者按 F12 就看得到。**這是設計上就這樣，不是疏失。**

它拿得到什麼，完全由你寫在資料庫裡的規則決定。規則寫對，這把金鑰公開了也拿不走別人的預約。規則沒寫，它什麼都拿得到。

另一把不一樣。它無視你寫的每一條規則。拿到它的人可以讀走、改掉、刪光所有資料。

金鑰與規則的分工

> 打包進網頁的那把金鑰本來就公開，擋住別人的是資料庫的規則。

第 2 題的答案

很多人第一次會答「可以」。**正確答案是「不能」。** 而你會在階段 13 親手證明它。

你在動手前寫下的答案，階段 13 會用兩個瀏覽器實際驗一次。

💡 那段指令為什麼那樣寫

剛才那段 Prompt 的四個問題，每一個都是刻意的：

問題	作用
第 1 題	逼它把答案落在「規則」上，而不是「金鑰要保密」
第 2 題	這個案例的縮小版。答案在階段 13 用兩個瀏覽器親手證明
第 3 題	要它舉錯誤訊息。兩層擋下來的訊息完全不同，看到訊息就知道是哪一層出問題
第 4 題	建立一個之後會一直用到的分類：**防守用的檢查 vs 體驗用的檢查**

第 3 題最容易被跳過。看得懂錯誤訊息屬於哪一層，你才知道要回頭改哪一個階段。

🧭 Checkpoint 1（做完階段 3）

口頭回答就好，三題都講得出來就往下走。

1. 那把會被打包進網頁的金鑰是公開的。那「只能看到自己的預約」是靠什麼保證的？
2. 那兩句錯誤訊息，各代表哪一層擋下來？
3. 階段 3 的第 2 題（額滿就藏按鈕，能不能防止超額），你當時是怎麼回答的？

> 第 3 題沒有標準答案，但**你要記得自己答了什麼**。階段 13 會回來問你。第 1、2 題答不出來就先不要往下，剩下十四個階段都建在這兩件事上。

階段 4　建立專案，並搬進案例一的設計規範

📖 讀完約 8 分鐘　🛠️ 動手約 30 分鐘

- **這一步一定要懂**：規範要**完全照抄，不要順便優化**。但規則裡寫死的檔案路徑與檔名，要重新對過一次。圓角一律 2px，所以輸入框是一條底線，不是圓角框。
- **這一步可以先不懂**：自動檢查的腳本怎麼寫、`.claude/settings.json` 怎麼把它掛上去。
- **做完後你要能用自己的話回答**：規範整段照抄過來，為什麼寫死路徑的那一類規則不能照抄？

⚖️ 動手前先想

這個案例會新增五種案例一沒有的東西。輸入框、載入中、錯誤訊息、成功訊息、額滿狀態。

每個工具都幫你準備好了大圓角輸入框和綠色的成功提示。照著用，這個網站就會和其他網站長一樣。

依照案例一的規範，它們應該長這樣。表裡的**火焰橘**是這個品牌的強調色。只用在溫度數字、剩餘名額很少、輸入框被點進去的時候。

一般做法	照這套規範	為什麼
大圓角輸入框	一條底線，點進去時變成火焰橘	圓角一律 2px，這條沒有例外
膠囊按鈕	底線 + 一個會往右移的箭頭	和案例一同一個元件
綠色勾勾 / 紅色浮出提示	一行字，需要強調時用火焰橘	用顏色編碼狀態是一般網站的慣例，不是這個品牌的語言
轉圈圈的載入圖示	「讀取窯次…」一行字	轉圈圈是反射動作，不是設計決策
紅色「額滿」標籤	灰色的「額滿」兩個字	額滿是狀態，不是警告

強調色的使用條件不放寬：多了表單，也只用在上面那三個地方。全站佔比仍然 < 5%。

🛠️ 動手做

🧰 這一步的產出

```
專案骨架（產生器建好的）
CLAUDE.md                  ← 案例一的設計規範，加上這個案例的五種新狀態
scripts/design-lint.mjs    ← 案例一的自動檢查
.claude/settings.json      ← 把檢查掛上去
src/styles/base.css        ← 顏色、字體、間距
src/layouts/Base.astro     ← 導覽列 + 頁尾
```

畫面上應該看到：一個空的首頁，但導覽列和頁尾的排版和 `_reference-vessel.html` 一致。

步驟一：建立專案

1. 在桌面建一個空資料夾，名字取 `kiln`
2. 開一個新對話，**Project folder** 選它，模式選 **Manual**
3. 貼上：

貼給 AI

你只需要修改：沒有。整段照貼。

```
幫我建立一個 Astro 專案：
- 建在目前這個資料夾裡，不要另外開子資料夾
- 用 minimal 範本（最乾淨的版本）
- 順便把需要的東西裝好
- 先不要接版本控制
- 不要加任何額外的整合套件

裝好之後再幫我裝跟 Supabase 溝通用的工具。

最後告訴我：CLAUDE.md 這個檔案是不是一個指向 AGENTS.md 的捷徑？
```

Claude 會跳卡片問你要不要允許它執行安裝指令。安裝屬於操作卡 2 的第二層，要你確認。卡片上寫的是安裝，就按允許。寫的是刪除檔案或改電腦的設定，先問它為什麼。

> ⚠️ 如果 Claude 想加上 `--add none` 這個參數，請它拿掉。那會噴一個 `Failed to add integrations` 錯誤。專案還是建得起來，但你會被嚇到。

最後那一題的答案應該是「是」。這代表 `CLAUDE.md` 是一個**捷徑**。真正的內容在 `AGENTS.md` 裡，改哪一個都一樣。

裝完跟 Claude 說「幫我把網站跑起來」，再從 **Views** 選單打開 **Browser** 面板。看到一個幾乎空白的頁面就對了，步驟二會把版面搬過來。

步驟二：搬設計規範與自動檢查

> **這個案例一律用課程附件裡的 VESSEL 規範。** 就算你做過別的題材，也用這一份，因為開窯預約是 VESSEL 的網站。
>
> 要用的兩個檔案都在桌面的 `case-03-kiln` 資料夾（階段 1 解壓縮的那個）：
>
> - `_reference-vessel.html`：案例一的成品
> - `CLAUDE.md`：裡面〈設計規範〉那一節就是 VESSEL 的規範
>
> **怎麼交給 Claude**：把檔案**直接拖進對話框**，或在對話框輸入 `@` 之後選它。

貼給 AI

你只需要修改：只有第一行：把那兩個檔案真的拖進對話框（或用 `@` 選檔案）。**其餘照貼**，尤其「完全照抄，不要順便優化」。

```
(把 _reference-vessel.html 和 CLAUDE.md 拖進對話框)

這是我上一個專案的成品和它的設計規範。請把三樣東西搬到這個新專案：

1. src/styles/base.css：顏色、字體角色、間距、圓角、進場動畫的參數，
   完全照抄，不要「順便優化」
2. src/layouts/Base.astro：導覽列與頁尾，同樣的排版語言
3. CLAUDE.md 裡〈設計規範〉那一節，整段搬過來。
   只搬這一節：前面的 Astro 英文說明和後面的〈上線關卡〉不要搬

設計規範裡已經有〈五種狀態〉一節，講輸入框、載入中、錯誤訊息、
```

成功訊息、額滿狀態。這五種在案例一的網站上都沒有。

請逐條用沒有程式背景的人聽得懂的方式告訴我：

「一般網站怎麼做」、「這套規範怎麼做」，以及為什麼。用表格回答。

步驟三：把自動檢查也搬過來

自動檢查一樣用附件的版本：桌面 `case-03-kiln` 資料夾裡的 `scripts/design-lint.mjs`。

貼給 AI

你只需要修改：沒有。整段照貼。多加的那兩條檢查規則是這個案例才需要的。

案例一有一個自動檢查，會在你每次改完檔案後檢查設計規範。

它在桌面的 case-03-kiln 資料夾裡：scripts/design-lint.mjs。

請把它也搬過來：scripts/design-lint.mjs、.prettierrc.json，

並在這個專案的 .claude/settings.json 掛上「改完檔案之後」的兩個檢查

（prettier 格式化和 design-lint）。安裝需要的工具。

附件的 scripts/ 裡還有 predeploy-check.mjs，settings.json 裡也有一個

「執行指令之前」的檢查。這兩個先不要搬，階段 17 會自己做一次。

搬完之後請多加兩條檢查規則，對應這個案例新增的狀態：

1. 輸入框不能有大於 2px 的圓角
2. 不能出現綠色或紅色的狀態顏色（成功、錯誤一律用文字表達）

裝完告訴我要怎麼確認它有生效。

自動檢查沒反應的話：在對話框輸入一次 `/hooks`，或者關掉對話重開一個。 Claude Code 只會監看「對話開始時就存在的資料夾」。

搬規範時最容易漏掉的一件事

規範裡有一類規則寫的是**檔案路徑**，例如案例一那條：

> 色票只能定義在 src/styles/tokens.css

但這個專案的色票檔叫 `base.css`。照抄的話，自動檢查會把你**正確的**檔案判成違規。接著 AI 會去「修」一個本來就沒問題的東西。

所以搬完之後要多做一步：

貼給 AI

> 剛剛搬過來的檢查腳本裡，有沒有哪一條規則寫死了檔案路徑或檔名？
>
> 列出來，並改成符合這個專案的實際結構。

搬規範的時候，規則的意圖可以照抄，規則裡的路徑與檔名要重新對過一次。

✅ 怎麼知道做對了

- [ ] 網站打得開，導覽列和頁尾的排版和 `_reference-vessel.html` 一致
- [ ] 顏色和 `_reference-vessel.html` 完全相同
- [ ] `CLAUDE.md` 裡有設計規範，而且包含新增的五種狀態
- [ ] 我說得出「輸入框在這套規範下為什麼不能用圓角框」
- [ ] **自動檢查有生效**：請 Claude 故意寫一條大圓角的樣式，對話裡要跳出一行違規訊息

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
找不到 `_reference-vessel.html`	附件沒解壓縮，或解壓縮到別的地方	重新把 `case-03-kiln.zip` 解壓縮到桌面
顏色跟 `_reference-vessel.html` 不太一樣	AI「順便優化」了	「請完全照抄原本的數值，不要調整」
自動檢查沒反應	資料夾是對話中途才建立的	輸入 `/hooks` 或重開對話
`Failed to add integrations`	指令裡有 `--add none`	請 Claude 拿掉這個參數

這一步失敗可以跳過嗎：設計規範可以先跳過，用預設樣式往下做。但**自動檢查建議一定要裝**，否則後面每一個有畫面的階段都會慢慢長回模板樣。

📖 為什麼這樣做

🎯 這一步的重點

案例一那份 `CLAUDE.md` 和自動檢查，搬到這個專案照樣擋得住違規的樣式。

🧭 這一步在做什麼

開一個新專案，並把案例一的三樣東西搬過來。設計規範（`CLAUDE.md`）、自動檢查、以及顏色與字體的設定。

案例一是一個沒有互動的品牌網站。這個案例有表單、有按鈕、有錯誤訊息。規範要在這五種新東西上也成立，才算搬成功。

🧩 它在建構流程的位置

這一步和資料庫那條線是平行的。先把版面與規範做好，資料等到階段 9 才接上去。

💡 那段指令為什麼那樣寫

步驟二那段 Prompt 裡有一句「**完全照抄，不要順便優化**」。

你和 AI 都會忍不住「改良」。結果是同一個品牌的兩個網站，顏色和圓角都對不起來。

步驟三那句是「**把自動檢查也搬過來**」。案例一花了一整個階段建立那份檢查，搬過來應該三分鐘就重新生效。

階段 5　設計兩張資料表

📖 讀完約 8 分鐘　🛠️ 動手約 25 分鐘

- **這一步一定要懂**：「一個人一窯只能預約一次」不是在前端檢查。它要用資料庫本身的規則保證。每個欄位都要說得出為什麼存在。
- **這一步可以先不懂**：SQL 語法、資料型別、兩張表怎麼連起來。**你只需要讀中文註解。**
- **做完後你要能用自己的話回答**：為什麼這條規則寫在資料庫裡就繞不過去？

⚖️ 動手前先想

階段 3 那一題再問一次：「一個人一窯只能預約一次」這條規則，要寫在前端還是資料庫？

先自己想：同一個人對同一窯送兩次預約，資料表裡會多出幾列？

做完這一步之後，不讀 SQL 也能確認，請 Claude 指給你看：

貼給 AI

```
請指出 schema.sql 裡是哪一行保證「一人一窯只能預約一次」。
同一個人送第二次時，資料庫會回什麼錯誤訊息？
```

答案在〈📖 為什麼這樣做〉，做完再去對照。

🛠️ 動手做

🧰 這一步的產出

```
supabase/schema.sql    ← 兩張表（權限留到階段 6）
```

畫面上沒有變化。這一步只寫檔案，還沒有貼到 Supabase 上。

📋 貼給 AI

貼給 AI

你只需要修改：只有「需求是」那四行（如果你做的是別的題材）。**下面四個要求、中文註解那句、以及最後「先不要寫權限」照貼。**

設計這個需求的資料表。需求是：

- 工作室每次柴燒開窯，開放少數名額讓人到現場看
- 每一窯有：窯次編號、開窯時間、名額、最高溫、用的土、用的釉、一段說明
- 登入的使用者可以預約，一個人一窯只能預約一個名額
- 使用者可以取消自己的預約

請給我 `PostgreSQL` 的建表語法，寫進 `supabase/schema.sql`，並且：

1. 只用最少的表。如果你想加第三張表，請先說服我為什麼非要不可
2. 「一個人一窯只能預約一次」這件事，要用資料庫本身的規則來保證，
   不要只在前端檢查。請說明你用哪一種規則、它在什麼情況下會擋下來
3. 使用者不要自己建一張表，直接用 `Supabase` 內建的帳號資料
4. 每一個欄位都要說明「為什麼需要它」。想不出理由的欄位就刪掉

我沒有程式背景，請在每一段 `SQL` 上面用中文註解說明它在做什麼。

這一步先不要寫權限，權限下一步再說。

最後應該長這樣

```
firings（窯次）                    reservations（預約）
├ kiln_no      第幾窯               ├ firing_id  ┐ 這筆預約是哪一窯
├ opens_at     開窯時間             ├ user_id    ┘ 誰預約的
├ capacity     名額                 ├ created_at 什麼時候約的
├ peak_temp    最高溫               └ 「同一人同一窯只能有一筆」的規則
├ clay / glaze 土與釉
└ note         說明
```

> **你要看的是什麼**：右邊那張表的每一列，都在說「**誰**預約了**哪一窯**」。左邊那張表是窯次本身的資料。兩張表靠 `firing_id` 這個欄位連起來。
>
> 右下角那條規則是這一步的重點。它寫在資料庫裡，只要資料是透過這個資料庫寫入的，就**繞不過去**。

✅ 怎麼知道做對了

- [ ] 只有兩張表
- [ ] 「一人一窯一次」是資料庫的規則，不是前端檢查
- [ ] 我可以指著每一個欄位說出它為什麼存在
- [ ] 每一段 SQL 上面都有中文註解，而且我讀得懂

前兩項不用讀 SQL，問 Claude 就好：「這份 `schema.sql` 建了幾張表？『一人一窯只能約一次』是寫在資料庫裡，還是要靠前端擋？」

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
它建了三張、四張表	想得太遠	「請用兩張表重寫，多的那些請說服我為什麼非要不可」
它另外建了一張使用者表	沒照第 3 點做	「請直接用 Supabase 內建的帳號資料，不要自己建使用者表」
註解是英文的	沒照最後一句做	「請把所有註解改成繁體中文」
欄位很多但我看不懂	想太多	「請逐一告訴我每個欄位為什麼需要，說不出理由的刪掉」

這一步失敗可以跳過嗎：不行。後面每一步都建在這兩張表上。

📖 為什麼這樣做

🎯 這一步的重點

說得出每一個欄位為什麼存在。說不出來的就刪掉。

🧭 這一步在做什麼

這一步要決定兩件事：這個網站需要哪幾張表、每張表有哪些欄位。不用會寫程式，但要想清楚。

> **名詞**：`.sql` 是資料庫看得懂的指令檔。這個案例的 `schema.sql` 放的是建表和權限的指令。你需要知道到什麼程度：知道你會把它整份貼到 Supabase 的網頁上按一次 Run。裡面的語法不用讀。

🧩 它在建構流程的位置

判斷 ──▶ 【兩張資料表】 ──▶ 兩層權限 ──▶ 示範資料 ──▶ 驗證

這是資料那條線的起點。 表的欄位決定後面權限規則要保護什麼，所以這裡改一次，後面全部要跟著改。

前端還是資料庫

〈⚖️ 動手前先想〉那一題的答案是**資料庫**。寫在前端的檢查，只要有人打開開發者工具就能跳過。寫在資料庫裡的規則不一樣，**就算有人繞過網頁直接送請求也擋得住**。

同一個人對同一窯送兩次預約，資料表裡只會多出一列。第二次會被資料庫擋下，原文是 `duplicate key value violates unique constraint`。階段 11 會在畫面上看到它被翻成「你已經預約過這一窯了」。

這個案例的主線在這裡第一次變成一行實際的 SQL。

💡 那段指令為什麼那樣寫

剛才那段 Prompt 裡的四個要求，每一個都對應一個常見錯誤：

要求	防止什麼
只用最少的表	一開始就分太細是最常見的錯誤。這個需求兩張表就夠，多一張都是負擔
用資料庫本身的規則保證	前端檢查繞得過去
使用者不要自己建表	Supabase 已經有帳號系統，自己再建一張會兩邊對不起來
每個欄位都要說明為什麼需要它	說不出理由的就刪掉

最後還有一句「請在每一段 SQL 上面用中文註解」。沒有註解，三個月後你打開這份檔案，會完全不知道每一段在做什麼。

階段 6　加上兩層權限：能不能碰這張表，和能碰哪幾列

📖 讀完約 12 分鐘　🛠️ 動手約 30 分鐘

> - **這一步一定要懂**：權限有兩層，兩層都要對。開了第二層（RLS）之後預設是**全部拒絕**，每多寫一條政策，就多放行一種讀取或寫入。以及剩餘名額為什麼前端算不出來。
> - **這一步可以先不懂**：政策的語法、security definer 函式怎麼寫。認得這兩個字就夠了。
> - **做完後你要能用自己的話回答**：那兩句錯誤訊息，各代表哪一層擋下來？

⚖️ 動手前先想

> **名詞：GRANT** 是資料庫發權限的指令，管的是「這個身分能不能碰這張表」。
> 你需要知道到什麼程度：看到 `permission denied for table` 就知道是它擋的。語法不用背。

> **名詞：RLS（列層級安全）** 是第二層權限，管的是「這個身分能碰這張表的哪幾列」。
> 你需要知道到什麼程度：知道它一打開就是全部拒絕，你要一條一條把該放行的寫回去。

> **名詞：政策（policy）** 是 RLS 底下的一條具體規則，例如「只能看自己的預約」。
> 你需要知道到什麼程度：正文說「三條政策」，指的是預約表的三條。只能看、只能新增、只能取消自己的預約。窯次表另外有一條「任何人都能看窯次」，不算在這三條裡。「資料庫的規則」是總稱，還包含 GRANT 和階段 5 那條「一人一窯一次」。

> **名詞：security definer** 是資料庫函式的一種設定。設了它，函式用建立者的權限執行，讀得到呼叫的人讀不到的資料。
> 你需要知道到什麼程度：認得這兩個字，因為它會出現在 AI 產出的 SQL 裡。怎麼寫不用會。

這個數字前端算不算得出來？

這一步有一個矛盾：

> 使用者看不到別人的預約（規則擋住了），但畫面上要顯示「還剩幾個位子」。

先自己想一下：這個數字前端算得出來嗎？算不出來的話，該由誰來算？答案在〈📖 為什麼這樣做〉，做完再去對照。

做完這一步之後再想一題：回傳剩餘名額的那個函式，有沒有哪一欄看得出是誰預約的？

怎麼看：在 SQL Editor 執行 `select * from firings_with_seats();`。參考成品的欄位只有窯次本身的資料和 `seats_left`，沒有 `user_id`，也沒有 email。也可以貼這段請 Claude 列出來：

貼給 AI

```
請列出 firings_with_seats 回傳的每一個欄位，並標出哪幾欄是從 reservations 來的。
除了剩餘名額之外，有沒有任何一欄看得出是誰預約的？
```

🛠️ 動手做

🧰 這一步的產出

```
supabase/schema.sql    ← 補上兩層權限、三條政策、一個計算函式
```

你自己的網站畫面沒有變化。但這一步做完，Supabase 的 **Table Editor** 裡會出現兩張空的表。那是你看得到的成果。

📋 貼給 AI（6-1：兩層權限）

貼給 AI

你只需要修改：沒有。整段照貼。五個要求一條都不要刪。尤其第 4 點（明確收回權限），那是階段 12 才會顯現價值的一條。

```
現在加上權限。分兩層寫，並在中文註解裡說明每一層在管什麼。

第一層（能不能碰這張表）：
- 沒登入的訪客：只能看窯次
- 登入的使用者：可以看窯次，可以看 / 新增 / 刪除自己的預約
- 訪客連預約那張表都不該碰得到

第二層（能碰哪幾列）：
- 窯次：任何人都能看
```

- 預約：只能看自己的、只能用自己的身分新增、只能刪自己的

五個要求：

1. 規則的名稱請用中文，讓不懂 SQL 的人也看得懂它在講什麼
2. 這份 SQL 要能重複執行很多次都不出錯（我一定會改很多次）
3. 請說明「看得到哪幾列」和「寫進去的那一列要符合什麼條件」
   是兩個不同的設定，少了後者會怎樣
4. Supabase 的專案預設會把 public 的權限給訪客，
   所以除了「給權限」之外，也要明確「收回」不該有的權限。
   請說明為什麼這件事重要
5. 每一段都要有中文註解

📋 貼給 AI（6-2：那個算不出來的數字）

貼給 AI

你只需要修改：沒有。整段照貼。

現在有一個矛盾：

使用者看不到別人的預約（規則擋住了），
但畫面上要顯示「還剩幾個位子」。

這個數字前端算得出來嗎？如果算不出來，該怎麼辦？

請用沒有程式背景的人聽得懂的方式回答：

1. 為什麼這件事一定要在資料庫做
2. 你的解法會不會不小心洩漏「是誰預約的」
3. 這個解法有什麼風險，要注意什麼

然後才實作，並且用中文註解說明。

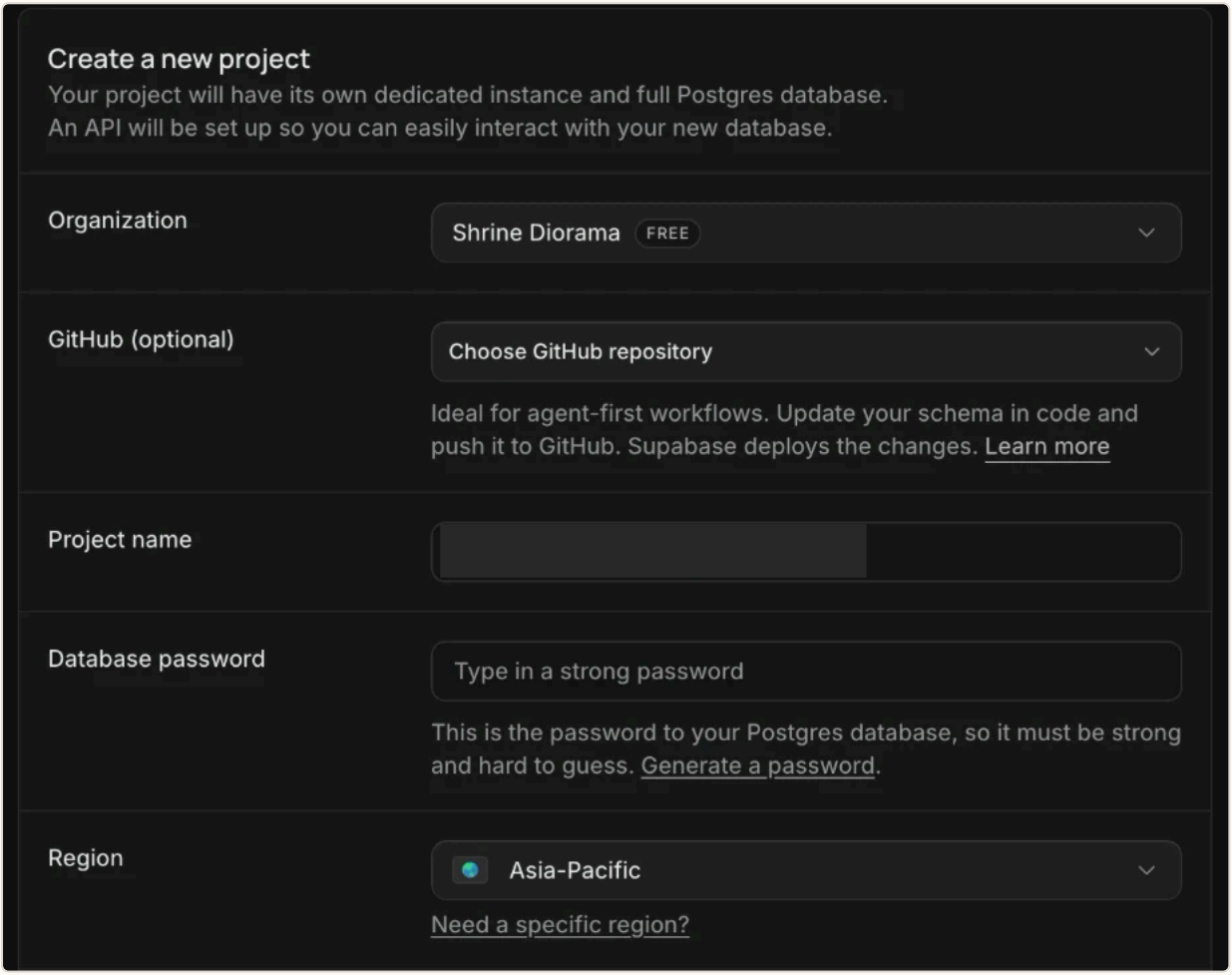
你自己做（6-3：貼到 Supabase 上）

現在把 SQL 真的執行一次。

1. 到 supabase.com 登入（〈你需要準備的東西〉那一步註冊的帳號），按 **New project**。它會要你填專案名稱、一組資料庫密碼、還有地區。密碼隨手設一組記下來（這個案例後面不會再用

到它，但還是記著）。地區選離台灣近的，例如 Singapore 或 Tokyo。選單裡只有 Asia-Pacific 這種大區的話，就選它。

2. 專案要建一到兩分鐘。建好之後，左側選單找到 **SQL Editor** → 右上角 **New query**
3. 把 `supabase/schema.sql` **整份**複製貼進去 → 按 **Run**
4. 下方的訊息區會顯示 **Success**。**再按一次 Run**，應該還是 Success（這就是「可以重複執行」）。第二次出現 NOTICE 或 skipping 是正常的，只有 ERROR 才是錯
5. 到左側 **Table Editor**，看得到 `firings` 和 `reservations` 兩張表。裡面還是空的，資料要到階段 7 才放



第 1 步的表單。畫面取自另一個練習專案，Organization 和你的不同，Project name 已遮掉。要填的是 Project name、Database password、Region 三欄。

✅ 怎麼知道做對了

- [ ] `schema.sql` 貼到 Supabase 執行兩次都不出錯
- [ ] 我說得出第一層和第二層各管什麼
- [ ] 我說得出那兩句錯誤訊息分別代表哪一層

- [ ] 我解釋得出為什麼「剩餘名額」要請資料庫算
- [ ] 每一條政策的名稱我都看得懂（因為是中文）

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
第二次 Run 出現 NOTICE 或 skipping	正常。這是說某樣東西已經建過了，所以跳過	不用處理。只有 ERROR 才是錯
第二次 Run 出現 ERROR	SQL 不能重複執行	「這份 SQL 第二次執行會出錯，請改成可以重複執行的寫法」
`permission denied`	第一層沒開	「請補上 GRANT」
什麼資料都讀不到，但沒有錯誤	開了 RLS 但沒寫政策	**這是正常行為，不是壞掉。** 回頭確認三條政策和窯次表那一條都寫了
政策名稱是英文的	沒照第 1 點做	「請把政策名稱改成繁體中文」
看不懂 Supabase 的介面	正常	截圖貼給 Claude：「我在這個畫面，SQL Editor 在哪裡？」

這一步失敗可以跳過嗎：不行。這是這個案例的核心。

📖 為什麼這樣做

🎯 這一步的重點

理解權限有兩層，而且**兩層都要對**。

🧭 這一步在做什麼

權限分兩層寫，兩層管的事不一樣。先看沒設好的後果：

這一層	管什麼	沒設好會怎樣	擋下來時的訊息
第一層　GRANT	能不能碰這張表	沒開，你連這張表都讀不到	`permission denied for table reservations`
第二層　RLS（列層級安全）	能碰這張表的哪幾列	沒開，這張表每一列都讀得到	`new row violates row-level security policy`

第二層漏掉的後果要講清楚。第一層開了、第二層沒開，任何登入的人都讀得到整張 `reservations`：誰約了哪一窯、什麼時候約的，全部看得到。這是最常見的資料外洩，而且畫面上

完全正常。

先自己想：你用 A 登入，看得到 B 的預約嗎？只用一個帳號試，永遠看不出來。

怎麼試：要兩個身分。階段 8 的測試 6 會換成 B 去數預約，應該是 0 筆。第二層沒開的話，B 會數到 A 那一筆。

寫完 6–1 之後，請 Claude 列給你看：

貼給 AI

```
請列出 schema.sql 裡每一張表有沒有開 RLS（列層級安全），
以及每一張表各有哪幾條政策，每一條放行誰做什麼。
```

上面那兩句錯誤訊息，就是階段 3 抄下來的那兩句。之後畫面壞掉時，看訊息就知道是哪一層出問題。

🧩 它在建構流程的位置

兩張資料表 ⟶ 【兩層權限】 ⟶ 示範資料 ⟶ 驗證

這是這個案例的核心，也是最容易寫錯的地方。

一個一定要記住的方向

> 開了第二層（RLS）之後，預設是全部拒絕。

所以每一條政策都是在**放行**，讓某些人看得到某幾列。政策寫得越多，放行的範圍越大。

這個方向搞反的話，會發生兩種狀況：

1. 你寫出一堆政策，以為每一條都在擋人，其實每一條都在放行
2. 你開了 RLS 但一條政策都沒寫，畫面變成「什麼都讀不到」，而你的第一反應通常是「Supabase 壞了」，**其實是它正常運作**

「現在是誰」這件事，前端說了不算

資料庫要判斷「這筆預約是不是你的」，得先知道你是誰。

> **名詞：通行證（JWT）** 是你登入之後拿到的一張有防偽的證明。每次跟資料庫要資料都會帶上。
> 你需要知道到什麼程度：知道資料庫是從它讀出「現在是誰」。防偽怎麼算可以不管。

前端偽造不了，只要簽發用的密鑰沒有外流，改一個字元就對不上。

剩餘名額為什麼前端算不出來

〈⚖️ 動手前先想〉那一題的答案是**算不出來**。前端手上根本沒有那些預約資料，怎麼算都算不出來。

正確解法是**請資料庫幫你算**，用一個設了 security definer 的函式。它讀得到整張預約表，但只回你一個數字。

風險也要知道：這個函式權限比較大，所以你要確定它只回傳數字。寫錯的話（例如讓它回傳整份預約名單），那就是外洩。

💡 那段指令為什麼那樣寫

6–1 那段 Prompt 有兩個容易忽略的要求：

「看得到哪幾列」和「寫進去的那一列要符合什麼條件」是兩個不同的設定。

- 「看得到哪幾列」管的是**讀取和刪除**
- 「寫進去的那一列要符合什麼條件」管的是**新增**

少了後者，任何人都可以幫別人建立預約。這是真實會發生的漏洞，而且從畫面上完全看不出來。

先自己想：預約那一列的 `user_id` 是誰填的？有人送一筆 `user_id` 寫成別人的預約，資料庫會收下嗎？

怎麼試：畫面上看不出來，要靠測試。階段 8 的測試 5「A 用 B 的身分寫入」就是在試這件事，預期被第二層擋下。

貼給 AI

```
請檢查 schema.sql 裡每一條管新增的政策，有沒有寫「寫進去的那一列要符合什麼條件」
（with check）。
沒有寫的列出來，並說明誰可以替誰寫入什麼。
```

另一個要求是「這份 SQL 要能重複執行很多次都不出錯」。

> 這份檔案會被跑很多次，改一次就跑一次。不能重複執行的 SQL，第二次貼上就會整頁錯誤訊息。

階段 7　產生五筆示範窯次

📖 讀完約 4 分鐘　🛠️ 動手約 10 分鐘

- **這一步一定要懂**：開窯時間要用「相對於現在」的寫法。寫死成固定日期，過幾個星期首頁就會變空白。另外要有一筆名額 8 的資料，階段 8 的測試會用到。
- **這一步可以先不懂**：SQL 怎麼寫、怎麼讓它重複執行不會產生重複資料。你只需要讀中文註解。
- **做完後你要能用自己的話回答**：沒有示範資料的話，階段 8 的測試為什麼全過也沒證明什麼？

🛠️ 動手做

🧰 這一步的產出

```
supabase/seed.sql    ← 五筆示範窯次
```

Supabase 的資料表裡會有五筆資料。畫面上還沒有變化（畫面要到階段 9 才接）。

📋 貼給 AI

貼給 AI

你只需要修改：只有第 3 點的內容描述（換成你的題材）。**第 1、2、4、5 點照貼**。相對時間與「第 47 窯、名額 8」都是後面測試要用的。

```
建立 supabase/seed.sql，放五筆示範窯次資料。

要求：
1. 開窯時間用「相對於現在」的寫法（例如「現在 + 9 天」），
   不要寫死成固定日期。請說明為什麼
2. 五筆的名額不要一樣。其中一筆是第 47 窯、名額設成 8，另一筆名額設成 4
3. 每一筆的說明文字要具體：溫度、用什麼土、什麼釉、
   以及一句「只有真的做過柴燒的人才寫得出來的觀察」
```

4. 不要寫成行銷文案
5. 這份也要能重複執行不出錯

用中文註解說明每一段。

你自己做：把 seed.sql 貼到 Supabase 執行

到 Supabase 的 **SQL Editor** → **New query** → 把 `supabase/seed.sql` 整份貼上 → **Run**。

然後到左側的 **Table Editor** → 選 `firings`，應該看到五筆資料。

✅ 怎麼知道做對了

- [ ] Supabase 的 `firings` 表裡有五筆資料
- [ ] 每一筆的開窯時間都在未來
- [ ] 有一筆的名額是 8
- [ ] 我說得出為什麼時間不能寫死

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
執行出錯，說找不到表	`schema.sql` 沒跑成功	回階段 6，先把 `schema.sql` 跑一次
重複執行變成十筆，或第二次執行出現 `duplicate key`	沒有防重複	「請改成可以重複執行不會產生重複資料、也不會報錯的寫法」
開窯時間是去年	寫死了日期	「請改用『相對於現在』的寫法」

這一步失敗可以跳過嗎：不行。沒有資料，階段 8 之後的測試全部沒得測。

📖 為什麼這樣做

🎯 這一步的重點

理解為什麼**沒有資料就測不出規則有沒有生效**。

🧭 這一步在做什麼

資料表建好了，但裡面是空的。這一步放進五筆假的窯次資料，讓後面的每一步都有東西可以測。

> **名詞：seed（示範資料）** 是一批假資料，讓剛建好的資料庫有東西可以測。
> 你需要知道到什麼程度：知道它是示範用的假資料，只放在練習用的 Supabase 專案裡。

🧩 它在建構流程的位置

兩層權限 ⟶ 【示範資料】 ⟶ 驗證規則

沒有這一步，階段 8 的測試會全部通過但什麼都沒證明。因為根本沒有資料可以被擋。

💡 那段指令為什麼那樣寫

有一個容易踩的地方：**時間要用「相對於現在」的寫法**。

把開窯時間寫死成一個日期，過幾個星期那些場次就變成過去式。而首頁只列未來的場次。你會看到一個空白的首頁，然後以為是政策寫錯了。

另外要指定一筆名額 8 的資料。階段 8 的測試會替 A 約一筆，再驗「還剩 7 個位子」。

階段 8　在自己電腦上驗證規則真的擋得住

📖 讀完約 12 分鐘　🛠️ 動手約 45 分鐘

- **這一步一定要懂**：測試前**一定要換身分**，因為管理員無視所有規則。測試要會自己印 ✓ / ✗，你才不用讀懂它。
- **這一步可以先不懂**：替代品（stub）補了哪兩樣、PostgreSQL 怎麼裝。**裝不起來可以從 8–3 開始跳過**，但 8–2 的 `rls-test.sql` 一定要產生，階段 12 要用。
- **做完後你要能用自己的話回答**：七項全過，為什麼不一定代表規則有效？

⚖️ 動手前先想

這個規則有沒有真的生效？

依據只有一個：**有沒有換一個身分實際測過？**

你平常連進資料庫用的是**管理員**的身分，而**管理員會無視所有規則**。

先自己想：不換身分就跑七項測試，結果會是什麼？

做完這一步之後，把換身分那一行註解掉，看哪幾項從 ✓ 變成 ✗。這是驗收的其中一項，一定要做。

🛠️ 動手做

🧰 這一步的產出

```
supabase/_local-auth-stub.sql    ← Supabase 的替代品，只在自己電腦上用
supabase/rls-test.sql            ← 七項測試，會自己印 ✓ 或 ✗
```

畫面上沒有變化。你會在對話裡看到七行 ✓。

注意順序：先產生這兩個檔案，再裝資料庫，最後才載入。

📋 貼給 AI（8-1：做替代品）

貼給 AI

你只需要修改：沒有。整段照貼。**最後那段「絕對不可以貼到 Supabase 網站上」的警告不要刪**。那是這個階段唯一會弄壞專案的操作。

```
我要在自己電腦的 PostgreSQL 上測試這份 schema.sql，但不想裝 Docker。

請寫一份 supabase/_local-auth-stub.sql，補上 Supabase 特有的東西：

1. 帳號那張表（欄位要和真的一樣：id、email、created_at）
2. 「現在是誰」那個函式。本機沒有通行證，
   所以改成從連線設定裡讀一個手動指定的值
3. 「訪客」和「已登入使用者」兩個角色
4. 把帳號資料的使用權給這兩個角色

第 4 點很重要，請說明為什麼，少了它會發生什麼事、
而且畫面上的症狀會長什麼樣？

另外請在檔案最上面用醒目的中文註解寫清楚：
這份檔案只能在自己電腦執行，絕對不可以貼到 Supabase 的網站上。
```

📋 貼給 AI（8-2：寫一份會自己回報的測試）

貼給 AI

你只需要修改：沒有。**七項測試與五個要求全部照貼**，包含 `<A>` `<B>` 兩個佔位符。階段 12 會把同一份拿到正式環境跑。

```
寫 supabase/rls-test.sql，證明規則真的擋得住。七項測試：

1. 訪客看得到窯次（預期：看得到）
2. 訪客碰不到預約那張表（預期：被第一層擋下）
3. A 登入後預約成功，而且只看得到自己那一筆
4. A 重複預約同一窯（預期：被資料庫的規則擋下）
5. A 用 B 的身分寫入（預期：被第二層擋下）
6. 換成 B，看不到 A 的預約（預期：0 筆）
7. 但 B 仍然查得到「47 窯還剩 7 個位子」
```

五個要求：

1. 每一項都要自己印出 ✓ 或 ✗，讓我執行一次就看得到全部結果，
   不需要看懂 SQL 也知道有沒有過
2. 預期會失敗的那幾項，要擱下來，不能讓它中斷整個測試
3. 測試前一定要「換身分」。請用中文註解說明為什麼，不換會怎樣
4. 兩個測試用的使用者 id 請寫成明顯的佔位符 <A> 和 <B>，
   並在檔案最上方寫清楚「執行前要把這兩個換成真的 id」。
   （之後我會把同一份拿到正式環境跑）
5. 最後要復原，測試不留下任何資料

你自己做（8-3：裝一個資料庫）

Mac 最簡單的方式是下載 Postgres.app：

1. 下載後拖進「應用程式」資料夾
2. 打開它
3. 按 **Initialize**

全程用滑鼠，不用打任何指令。

> **名詞：psql** 是 PostgreSQL 附的指令工具，Claude 用它連上資料庫。
> 你需要知道到什麼程度：知道它的位置就好。Claude 找不到它的時候，告訴它 Postgres.app 的 psql 在 `/Applications/Postgres.app/Contents/Versions/latest/bin`。

Windows 的話，到 postgresql.org/download/windows 下載安裝程式。一路按下一步，記下你設定的密碼。

- 公司電腦通常要 IT 或系統管理員權限才能裝
- 安裝過程有一步選語系（Locale），選 C 或 English
- 裝不起來就跳過這一階段，用階段 12 代替

連線密碼不要貼在對話框。 對話內容會留在紀錄裡。Claude 需要密碼時，跟它說：

貼給 AI

請不要叫我把資料庫密碼貼在對話框。
請設定好讓我自己在終端機輸入（例如用 `pgpass`），並一步一步告訴我要做什麼。

不想處理密碼的話，直接跳過這一階段，用階段 12 代替。

裝完之後，把這句貼給 Claude：

貼給 AI

你只需要修改：沒有。整段照貼。

我剛剛裝好 `PostgreSQL`。幫我確認它有沒有正常運作，
以及你能不能連得上。連不上的話告訴我要調什麼。

📋 貼給 AI（8–4：載入並執行）

貼給 AI

你只需要修改：沒有。整段照貼。**尤其最後那句「任何一步失敗就停下來，不要自己想辦法繞過」。**

我的電腦上已經有 `PostgreSQL` 了。請幫我做五件事，
每一步做完都告訴我結果：

1. 建一個叫 `kiln_test` 的空資料庫（測試用的，弄壞也沒關係）
2. 把 `supabase/_local-auth-stub.sql` 放進去（Supabase 的替代品）
3. 把 `supabase/schema.sql` 放進去（我的資料表和規則）
4. 把 `supabase/seed.sql` 放進去（五筆示範窯次）
5. 建立兩個測試用的使用者，把 `rls-test.sql` 裡的 `<A>` `<B>` 換成他們的 `id`，
   然後執行測試

任何一步失敗就停下來告訴我，不要自己想辦法繞過。

最後那一句很重要。 資料庫的錯誤常常是環境問題：沒裝好、路徑不對。讓 AI 繞過去只會讓問題藏得更深。

執行之後應該看到

```
測試 1  ✓  訪客看得到 5 個窯次
測試 2  ✓  訪客碰不到 reservations（第一層擋下）
測試 3  ✓  A 預約成功，看得到 1 筆
測試 4  ✓  重複預約被資料庫的規則擋下
測試 5  ✓  冒用他人身分被第二層擋下
測試 6  ✓  B 看得到 0 筆預約（A 的那筆應該看不到）
測試 7  ✓  B 查到 47 窯還剩 7 個位子（預期 7）
```

📋 貼給 AI（8–5：親手證明測試是有效的）

這一項一定要做。 它要確認的是：**測試有沒有真的換成使用者的身分在跑。**

貼給 AI

你只需要修改：沒有。整段照貼。

```
請幫我做一個實驗：
把 rls-test.sql 裡所有「換身分」的那幾行（set local role）暫時註解掉，
然後重新執行一次。
我要看測試 6 的結果變成什麼樣。

我看完之後你再把它改回來，並重新執行確認七項都是 ✓。
```

你應該看到：測試 2、5、6 從 ✓ 變成 ✗。其中測試 6 會變成 B 看得到 A 的預約（不再是 0 筆）。這時候 ✗ 後面寫的原因不用理，原因就是沒有換身分。

為什麼：因為沒換身分，你是用管理員的身分在跑，而管理員無視所有規則。**這時候還是 ✓ 的那幾項，什麼都沒證明。**

✅ 怎麼知道做對了

- [ ] 七項全部是 ✓
- [ ] 把「換身分」那幾行註解掉之後，測試 2、5、6 變成 ✗
- [ ] 改回來之後又全部是 ✓

- [ ] 我說得出為什麼用管理員身分測不出問題
- [ ] `rls-test.sql` 裡有 `<A>` 和 `<B>` 兩個佔位符

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
Postgres.app 裝不起來	公司電腦限制、硬碟空間	確認 8–2 的 `rls-test.sql` 已經產生，再**跳到階段 9**，用階段 12 的雲端測試代替
錯誤訊息裡有 `LC_ALL` 或 `multithreaded`	資料庫的語系設定和電腦對不上	跟 Claude 說：「請用 LC_ALL=C 重新啟動資料庫。」
Claude 連不上資料庫	連線設定不對	把它的錯誤訊息整段貼回去，再加一句：「請先不要修改任何東西。先告訴我最可能的三個原因，以及你準備先檢查哪一個。」
測試 3 就失敗了	替代品的第 4 點沒做	「測試 3 失敗，請檢查帳號資料的使用權有沒有給那兩個角色」
明明登入了卻寫不進去	同上	同上。這就是 8–1 第 4 點講的那個症狀
測試檔裡沒有 `<A>`	8–2 的第 4 點沒做	「請把測試用的 id 改成 `<A>` 和 `<B>` 佔位符」

這一步失敗可以跳過嗎：可以。 裝不起來就直接到階段 9，階段 12 會在正式環境做同樣的事。8–2 的 `rls-test.sql` 還是要先產生，階段 12 跑的就是它。但裝得起來就建議做。之後每改一次政策，你在這裡三十秒就能驗完。

📖 為什麼這樣做

🎯 這一步的重點

在**沒有網路、沒有 Docker** 的情況下，驗證自己寫的規則是對的。

🧭 這一步在做什麼

規則寫完之後，你怎麼知道它真的有效？

看起來對不算數。 你要真的換一個身分試試看，看它擋不擋得住。

不換身分的話，管理員讀得到每一列。測試 2、5、6 會變成 ✗，其他幾項照樣 ✓。**那幾個 ✓ 什麼都沒證明。**

這一步會在你自己的電腦上裝一個資料庫，把規則跑一遍。好處是改一次規則就能立刻驗一次。不用等網路、不用開瀏覽器，**也不用擔心把正式資料弄壞**。

🧩 它在建構流程的位置

```
示範資料 ──▶ 【驗證規則】 ──▶ 接上畫面
```

規則沒驗過就接畫面，你會把「看起來正常」誤當成「安全」。

這一步可以跳過

這是這個案例唯一需要在電腦上另外裝東西的地方（除了 Node.js）。

> **如果裝不起來**（公司電腦有限制、硬碟不夠、時間不夠），8–1、8–2 照做（只產生檔案），從 8–3 開始跳到階段 9，之後用階段 12 的雲端測試代替。效果一樣，只是每次改規則都要重新上傳，慢一點。
>
> **不要卡在這裡。**

為什麼可以不用 Docker

Supabase 的 SQL 裡只有兩個東西是它特有的。**帳號那張表**，以及**那個判斷「現在是誰」的函式**。除此之外全部是標準的 PostgreSQL。

所以在自己電腦的 PostgreSQL 上**做一個替代品**，補上這兩樣。你寫的規則就能原封不動跑起來。

> **名詞：stub（替代品）** 是一份假的、但長得一樣的東西，用來頂替真的那個。
> 你需要知道到什麼程度：知道 `_local-auth-stub.sql` 頂替的是 Supabase 的帳號系統。它只能在自己電腦上跑。

💡 那段指令為什麼那樣寫

產生替代品那個 Prompt 的第 4 點是真的會踩的地方，而且症狀會騙人。

政策會呼叫「現在是誰」那個函式，用的是**發出查詢的那個人**的身分。那個身分如果沒有權限讀帳號資料，政策就評估失敗。你在畫面上看到的症狀卻是「**明明登入了卻寫不進去**」。你會往登入那邊找，但問題在權限設定的另一個角落。

另外有一個一定要標清楚的警告。 那份替代品的 SQL 只能在自己電腦執行，絕對不可以貼到 Supabase 的網站上。真的 Supabase 已經有帳號系統，跑這份會蓋掉它。**這是會弄壞專案的操作。**

> **測試檔要用佔位符寫。** 階段 12 你會把同一份測試拿到正式環境跑，那時候要換成真的使用者 id。事先寫成明顯的佔位符，例如 `<A>`、`<B>`，到時候才知道要換哪裡。

🧭 Checkpoint 2（做完階段 8）

到這裡畫面上還是空的，**那是正常的**。你手上已經有兩張表、三條政策、五筆窯次，和七項會自己回報的測試。規則寫完了，也驗過了。

1. 為什麼測試前一定要換身分？不換會怎樣？
2. 「還剩幾個位子」為什麼不能在前端算？
3. 開了 RLS 卻一條政策都沒寫，畫面會變成什麼樣？那是壞掉嗎？

> 第 3 題答「Supabase 壞了」的話，回階段 6 看那段「一定要記住的方向」。

階段 9　把窯次資料接到畫面上

📖 讀完約 8 分鐘　🛠️ 動手約 30 分鐘

- **這一步一定要懂**：資料是**使用者的瀏覽器**直接去跟資料庫拿的，中間沒有伺服器。剩餘名額要用資料庫函式拿，不能自己算。
- **這一步可以先不懂**：`.env` 的機制、Astro 為什麼規定 `PUBLIC_` 開頭、請求怎麼發出去。
- **做完後你要能用自己的話回答**：這個網站沒有伺服器。那資料是什麼時候、在哪裡被拿到的？

🛠️ 動手做

🧰 這一步的產出

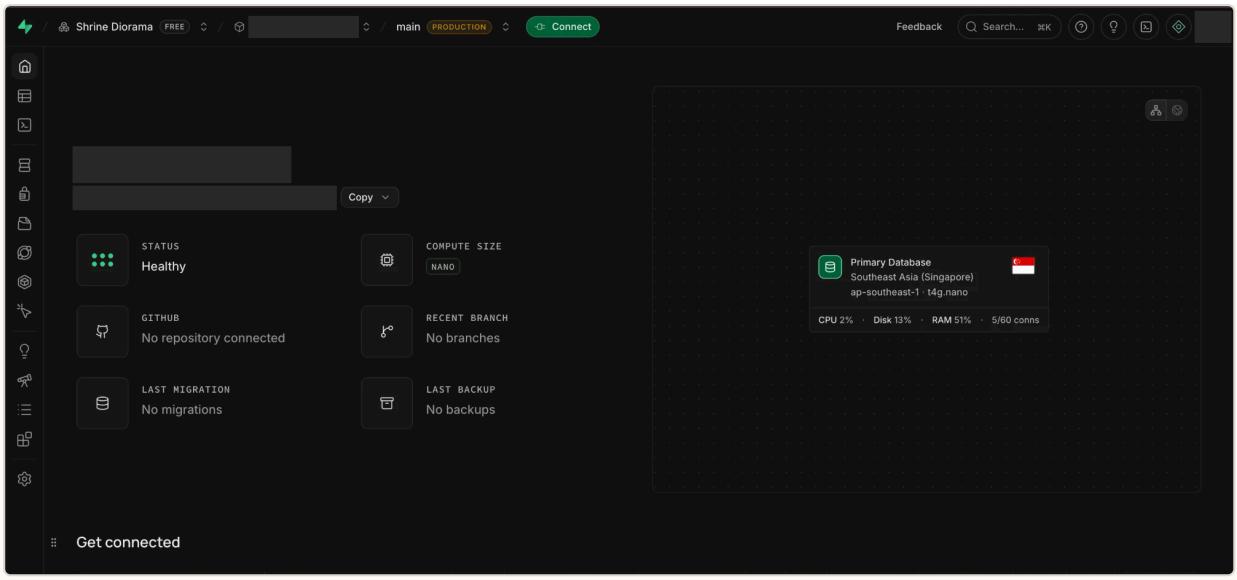
```
.env                        ← 放 Supabase 的網址和金鑰（不會進版本控制）
src/lib/supabase.js         ← 跟資料庫講話的設定
src/pages/index.astro       ← 場次列表
```

畫面上應該看到：五個窯次，每個都有剩餘名額。

步驟一：拿到兩個值

1. 在 Supabase 專案裡，左下角齒輪 **Settings** → **API Keys**（或直接點首頁下方「Get connected」那排最右邊的 **API Keys**）
2. 複製兩個值：
   - **Project URL**：首頁標題下面那一行就有，旁邊有 Copy 按鈕
   - **可以公開的那一把金鑰**：新專案是 **Publishable key**（開頭 `sb_publishable_`）。舊專案是 **anon public**（開頭 `eyJ`）。兩種都能用

> ⚠️ 絕對不要複製 `sb_secret_` 或 `service_role` 那一把。那是階段 3 講的那一把不能外流的金鑰，它會無視你寫的所有規則。
>
> 判斷方法很簡單：畫面上會提醒你「這一把不能外流」的，就是不能拿的那一把。



專案首頁。畫面取自另一個練習專案，專案名稱與網址已遮掉。左側選單最下面的齒輪就是 **Settings**。標題下面那一行旁邊是 **Copy**。

步驟二：建立設定檔

> **名詞**：**環境變數**是放在 `.env` 這個檔案裡的設定值，例如網址和金鑰。
> 你需要知道到什麼程度：知道它不會進版本控制。所以部署的時候，要在 Vercel 上再設一次（階段 17）。

請 Claude 在專案裡建一個叫 `.env` 的檔案，內容是下面兩行。等號右邊換成你的值，整段貼給它。

一律請 Claude 建立和改名，不要用記事本或文字編輯器自己建。 Windows 會把它存成 `.env.txt`，網站讀不到。 `.env` 是點開頭的檔案，在 Finder / 檔案總管預設看不到，所以之後要改名也請 Claude 做。

貼給 AI

你只需要修改：**等號右邊**兩個值，換成你剛剛複製到的網址與金鑰。**等號左邊的名稱一個字都不要改。**

```
請在專案根目錄建立 .env，內容是下面兩行，建好之後重新啟動網站：


PUBLIC_SUPABASE_URL=剛剛複製的 Project URL
PUBLIC_SUPABASE_ANON_KEY=剛剛複製的那把可以公開的金鑰
```

為什麼欄位名稱要 `PUBLIC_` 開頭：這是 Astro 的規定，有這個開頭的設定才會被送到瀏覽器。這個命名本身就在提醒你：它是公開的。

📋 貼給 AI

貼給 AI

你只需要修改：沒有。整段照貼。第 1 點（沒設定也不要壞掉）和第 3 點（剩餘名額要用資料庫函式拿）是這一步的重點。

```
接上資料庫。要求：

1. src/lib/supabase.js：建立連線。
   如果 .env 還沒填，不要讓網站壞掉，要讓畫面能顯示
   「尚未連上資料庫，請先設定 .env」，並說明要做什麼
2. 首頁列出未來的窯次，每一列顯示：窯次編號、開窯時間、
   還有幾天、最高溫、土、釉、說明、剩餘名額
3. 剩餘名額請用階段 6 那個資料庫函式拿，不要在前端自己算
4. 版面用橫列的排法，不要用卡片（沿用案例一的規範）
5. 讀取中顯示一行「讀取窯次…」，不要轉圈圈

我沒有程式背景，請在 JavaScript 裡也加上中文註解，
特別是「這一行為什麼要這樣寫」的部分。
```

✅ 怎麼知道做對了

- [ ] 首頁看得到五個窯次，每個都顯示剩餘名額
- [ ] 跟 Claude 說：「請把 .env 暫時改名成 .env.bak，我看完再請你改回來。」重新整理後往下捲到「近期窯次」那一段，畫面會說明「尚未連上資料庫」而不是白屏。看完請 Claude 改回來
- [ ] 排版和 `_reference-vessel.html` 一致：沒有卡片、圓角很小
- [ ] 用 Chrome 打開 Browser 面板網址列上的同一個網址，打開開發者工具 → Network，看得到一個往 Supabase 發的請求

最後一項一定要做。你會親眼看到資料是使用者的瀏覽器去拿的，不是伺服器準備好的。

不知道怎麼打開開發者工具的話：

貼給 AI

請一步一步告訴我怎麼打開瀏覽器的開發者工具、切到 `Network` 分頁，
然後找到那個往 `Supabase` 發的請求。我沒有做過。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
「尚未連上資料庫」	`.env` 沒填或填錯	檢查兩個值有沒有貼完整（金鑰很長，很容易少複製）
一片空白，什麼都沒有	沒有未來的場次	回階段 7，確認開窯時間是「相對於現在」
讀不到資料但沒有錯誤	開了 RLS 但政策沒生效	**這是正常行為不是壞掉。** 回階段 6 確認窯次表有「任何人都能看窯次」那一條
`permission denied`	第一層沒開	回階段 6 補 GRANT
剩餘名額都是 0 或空白	沒用那個資料庫函式	「請改用 firings_with_seats 函式拿剩餘名額」

這一步失敗可以跳過嗎：不行。這是第一次看到成果。

📖 為什麼這樣做

🎯 這一步的重點

理解「靜態網站也能有資料庫」，並且知道資料是**什麼時候**、**在哪裡**被拿到的。

🧭 這一步在做什麼

一般人以為「有資料庫的網站一定要有伺服器」。不一定。

這個網站的運作方式是：

```
使用者打開網頁
        ↓
瀏覽器載入一份寫死的 HTML（沒有任何資料）
        ↓
瀏覽器直接跟 Supabase 要資料          ← 這一步在使用者的電腦上發生
        ↓
資料回來，畫面才填上去
```

中間沒有任何伺服器。 所以這個網站可以丟到任何免費的靜態空間。

代價是使用者會先看到一瞬間的「讀取窯次…」。**不用伺服器就會有這一段等待。** 所以這個等待狀態也要照規範做：一行字，不是轉圈圈。



> **你要看的是什麼**：每一列是一個窯次，右邊有剩餘名額。 **那個數字不是前端算出來的**，前端看不到別人的預約，是資料庫幫你算好再回傳的。排版沿用案例一：沒有卡片、沒有圓角。

先問自己：上面這張截圖裡，「最高溫 1280°C」是什麼顏色？照設計規範，溫度要用火焰橘。



> **你要看的是什麼**：局部放大。比較「最高溫 1280°C」和旁邊「土」「釉」的顏色。

最高溫和「土」「釉」一樣是灰色。這張截圖裡就有這個錯。`.firing__spec span` 這條規則比 `.ember` 多一層選擇器，把顏色蓋掉了。

> **名詞：選擇器**是 CSS 用來指定「這條樣式套用在哪些元素上」的那一段。寫得越具體，優先權越高。例如 `.firing__spec span` 比 `.ember` 具體，兩條都設顏色時前者贏。
> 你需要知道到什麼程度：顏色和規範對不上時，知道可能是另一條更具體的規則蓋掉了。語法不用會。

CSS 原始碼看起來沒錯，要在瀏覽器裡量才看得出來。修正後，橘色只出現在兩個地方：最高溫，以及剩餘名額很少的時候。你的列表也量一次：

> 貼給 AI
>
> 請在瀏覽器裡量首頁列表「最高溫」那幾個字實際算出來的顏色，和 `base.css` 的 `--ember` 比對。
> 不一樣的話，告訴我是哪一條規則蓋掉的，先不要改。

🧩 它在建構流程的位置

> 驗證規則 ⟶ 【接上畫面】 ⟶ 登入

規則驗過了才接畫面。 這個順序是刻意的。

💡 那段指令為什麼那樣寫

第 1 點（沒設定也不要壞掉）對自學特別重要。

你一定會有一次把設定貼錯。畫面如果直接白屏加一堆紅字，你會卡住，而且不知道從何問起。

錯誤畫面會說人話，可以省下你半小時。這就是階段 1 你看到的那個畫面。

第 3 點是階段 6 的延續。 你會很想「把預約全部抓下來自己數就好了」。**抓不到，政策擋住了。** 這個限制是特意設計的。

階段 10　做不用密碼的登入

📖 讀完約 6 分鐘　🛠️ 動手約 30 分鐘

> - **這一步一定要懂**：登入改成寄一封信給使用者，網站裡沒有存任何一組密碼。Redirect URLs 沒設定的話，信裡的連結一定會被擋。
> - **這一步可以先不懂**：通行證怎麼被處理、登入狀態怎麼保存。使用者點信裡的連結回來之後，登入是工具自動完成的。
> - **做完後你要能用自己的話回答**：為什麼「能收到這個信箱的信」就足以證明你是你？

🛠️ 動手做

🧰 這一步的產出

```
src/pages/me.astro    ← 未登入顯示 email 表單；已登入顯示我的預約
```

畫面上應該看到：`/me` 這一頁有一個 email 輸入框；登入之後變成「我的預約」。

步驟一：先做一個設定，不然一定會失敗

> **名詞**：Redirect URLs 是 Supabase 的一份白名單。它登記「登入完可以把使用者送回哪些網址」。
> 你需要知道到什麼程度：知道沒登記過的網址一律被擋。本機網址和正式網址要各登記一次（階段 17）。

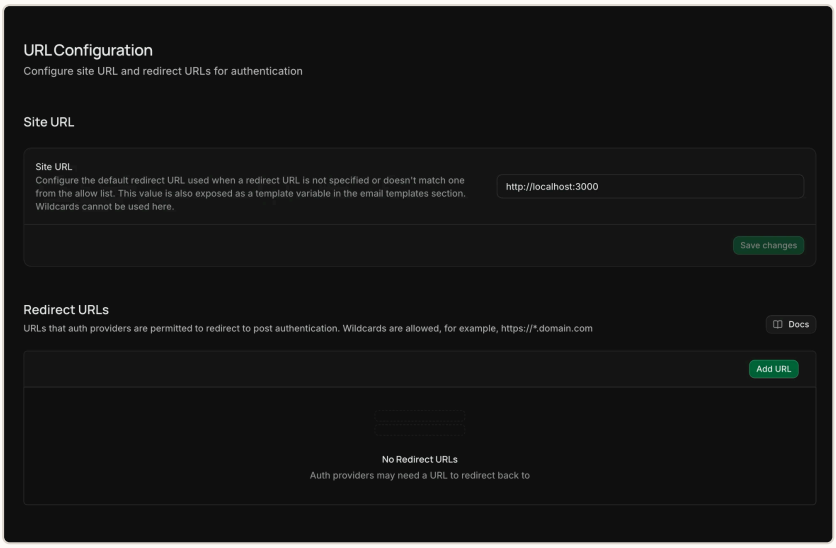
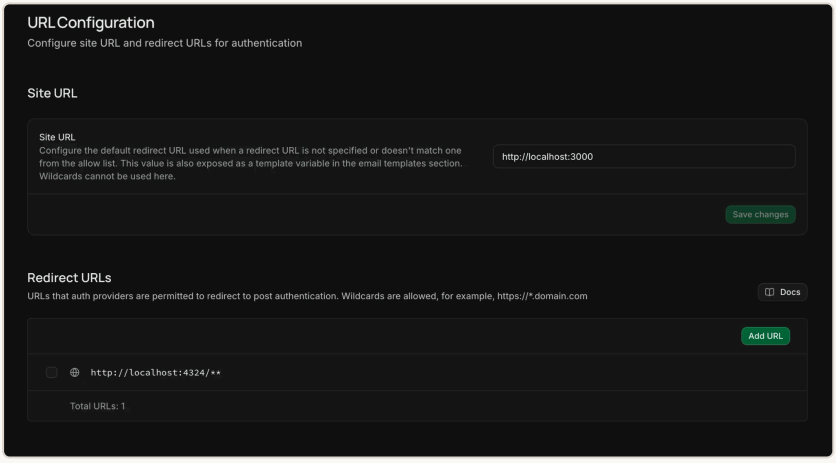
Supabase 網站 → 左側 **Authentication** → **URL Configuration** → 找到 **Redirect URLs** 那一區，按 **Add URL**，填進去這一行然後存檔：

```
http://localhost:4321
```

先看 Browser 面板網址列上的數字。如果不是 4321（例如階段 1 的附件伺服器還開著，新專案會改用 4322），就填那一個。

沒加的話，信裡的連結點下去會被擋掉。這是安全機制，不然這個連結會被拿去做釣魚。

兩張畫面取自另一個練習專案。

狀態	參考畫面
還沒加	 Redirect URLs 在下半部，**Add URL** 在那一區的右上角。上半部的 Site URL 這一步不用動。
加好一筆	 列表多一行，下面寫 Total URLs: 1。圖上那一行是那個練習專案的網址（port 4324），你填的是 `http://localhost:4321`。

> 這一步很多人會忘記，而且症狀很像「壞掉了」。

📋 貼給 AI

貼給 AI

你只需要修改：沒有。整段照貼。第 3 點不要簡化成「顯示成功」。

做登入。要求：

1. /me 這一頁：沒登入顯示 email 輸入表單，登入後顯示「我的預約」
2. 用 magic link，不要密碼
3. 寄出之後要明確告訴使用者「信寄到哪個信箱了、接下來要做什麼」，
   不要只顯示一個打勾
4. 使用者點信裡的連結回來時，要自動變成已登入狀態
5. 要有登出

設計上沿用案例一的規範：
- 輸入框是一條底線，不是圓角框
- 按鈕是「底線 + 箭頭」，不是膠囊
- 成功訊息是一行字，不要綠色勾勾

請在程式碼裡用中文註解說明每一段在做什麼，
特別是第 4 點是誰在處理、什麼時候發生。

✅ 怎麼知道做對了

- [ ] 用自己的 email 登入一次，收得到信，點完回到網站是已登入狀態
- [ ] 登出之後重新整理，還是登出狀態
- [ ] 輸入框是一條底線，不是圓角框
- [ ] 故意把 Redirect URLs 那個設定刪掉，再試一次，看到它失敗，然後加回去

最後一項是刻意的。先製造一次失敗，你之後遇到才認得出來。

信裡的連結會開在你電腦的預設瀏覽器，不是 Browser 面板，所以要在那個瀏覽器確認已登入。想在 Browser 面板登入，就把連結複製起來，貼到 Browser 面板的網址列。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
信裡的連結點下去被擋掉	Redirect URLs 沒加	Authentication → URL Configuration 補上
收不到信	跑到垃圾信件	檢查垃圾信件匣。還是沒有就等三分鐘
點了連結回來還是未登入	第 4 點沒處理好	「點連結回來之後還是未登入，請檢查網址上的通行證有沒有被處理」
一天寄太多封被限制	免費方案有寄信上限	等一小時，或換一個 email

這一步失敗可以跳過嗎：不行。階段 11 到 17 全部需要登入。

📖 為什麼這樣做

🎯 這一步的重點

理解「不用密碼的登入」怎麼運作，以及為什麼它比自己存密碼安全。

🧭 這一步在做什麼

這個網站不存密碼。流程是：

```
你輸入 email
      ↓
Supabase 寄一封信給你
      ↓
你點信裡的連結
      ↓
回到網站，已經登入了
```

> **名詞**：**magic link（免密碼登入）** 是把登入改成寄一封信。點信裡的連結就登入了。
> 你需要知道到什麼程度：知道它的前提是「能收到這個信箱的信，就足以證明你是你」。信怎麼寄、連結怎麼驗證由 Supabase 處理。

好處：

- **你不用學怎麼安全地保存密碼**，因為資料庫裡根本沒有那個欄位

- 使用者不用再記一組密碼
- 這個網站被入侵的話，攻擊者拿不到任何一組密碼



> **你要看的是什麼**：輸入框是一條底線，不是圓角框。游標點進去的時候，那條底線變成火焰橘。這是案例一規範裡「強調色只用於互動焦點」的實際樣子。按鈕是「底線 + 箭頭」，不是膠囊。

按下之後畫面會顯示「信寄到〈你的信箱〉了，這個分頁可以留著不用關」。**那句話說清楚接下來要做什麼，不是一個綠色勾勾。** 剛才那段 Prompt 第 3 點要求的就是這件事。

🧩 它在建構流程的位置

> 接上畫面 ⟶ 【登入】 ⟶ 預約與取消

寫入需要知道「現在是誰」，所以登入必須在寫入之前。

💡 那段指令為什麼那樣寫

第 3 點不能簡化。

免密碼登入最大的問題是使用者按完之後**不知道發生了什麼事**。信還沒到、跑到垃圾信件、以為壞了，都是這時候發生的。

使用者會看到一句「信寄到 you@example.com 了，這個分頁可以留著不用關」。這一句可以消掉大部分的困惑。

第 4 點其實不用自己寫，Supabase 的工具會自動處理網址上的通行證。但你要知道有這一步。壞掉的時候，你才知道要回去看網址上的通行證有沒有被處理掉。

階段 11　做預約與取消，並把錯誤翻成人話

📖 讀完約 8 分鐘　🛠️ 動手約 35 分鐘

- **這一步一定要懂**：資料庫的錯誤要翻成人話給使用者、原文留在 console 給你自己。額滿就藏按鈕**只是體驗，不是保證**。
- **這一步可以先不懂**：錯誤碼各代表什麼、畫面怎麼重新讀取資料。
- **做完後你要能用自己的話回答**：「額滿就不顯示按鈕」能不能真正防止超額預約？為什麼？

⚖️ 動手前先想

階段 3 那一題再問一次：這個檢查放前端還是資料庫？

下面那段 Prompt 的最後一題是這一句。「額滿就不顯示預約按鈕，能不能真正防止超額預約？」

這正好是階段 3 的第 2 題。先回想你在階段 3 寫下的答案，再貼 Prompt。階段 13 會用兩個瀏覽器驗證這個答案。

做這一步的時候邊做邊看。你加在畫面上的每一個檢查，有沒有哪一個擋得住繞過網頁的人？

🛠️ 動手做

🧰 這一步的產出

```
src/pages/firing.astro    ← 單一窯次的頁面 + 預約按鈕
```

畫面上應該看到：點首頁的某一窯進到詳細頁，可以預約、可以取消。

📋 貼給 AI

貼給 AI

你只需要修改：沒有。整段照貼。**最後那個問題（額滿藏按鈕能不能防超額）一定要留著**。階段 13 會用兩個瀏覽器驗證這一題的答案。

做預約流程。要求：

1. 點首頁的某一窯，進到那一窯的頁面
2. 已登入 → 顯示「預約這一窯」；未登入 → 顯示「請先登入」並帶連結
3. 預約成功後，畫面要立刻反映（名額少一個、按鈕變成「取消預約」）
4. 額滿時不顯示預約按鈕，改顯示一行灰色的說明

錯誤處理請特別做：

5. 如果使用者已經預約過這一窯，資料庫會回一個錯誤碼。
   不要把原始錯誤丟到畫面上，要翻成「你已經預約過這一窯了」。
   但原始錯誤要印在 `console` 裡，方便我除錯
6. 其他錯誤也要有一句人話，不要只顯示英文

最後請告訴我：第 4 點（額滿就不顯示按鈕）
能不能真正防止超額預約？為什麼？

✅ 怎麼知道做對了

- [ ] 預約成功後，名額立刻少一個
- [ ] 同一窯預約第二次，看到的是「你已經預約過這一窯了」而不是英文錯誤
- [ ] 原始錯誤有印在 console 裡（console 是瀏覽器開發者工具裡的一個分頁，不知道怎麼打開就問 Claude）
- [ ] 取消之後名額回來
- [ ] 我回答得出「額滿就不顯示按鈕能不能防止超額」，答案是不能

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
`new row violates row-level security policy`	第二層的新增政策沒寫對	回階段 6，檢查「寫進去的那一列要符合什麼條件」

你可能會看到	通常是因為	怎麼辦
明明登入了卻寫不進去	通行證沒帶上	「寫入時說我沒有權限，但我已經登入了，請檢查請求有沒有帶上登入狀態」
畫面上名額沒有立刻變	沒有重新讀取	「預約成功之後畫面沒有更新，請在成功之後重新讀一次剩餘名額」
看到一長串英文錯誤	第 5 點沒做	「請把資料庫的原始錯誤翻成人話，原文印在 console 就好」

這一步失敗可以跳過嗎：不行。階段 13 要用它來重現漏洞。

📖 為什麼這樣做

🎯 這一步的重點

完成整個流程，並學會**把資料庫的錯誤訊息翻成人話**。

🧭 這一步在做什麼

到目前為止都是「讀」。這一步要「寫」，而**權限寫錯的地方，通常要到寫入時才會出錯**。



你要看的是什麼：預約成功之後名額立刻少一個，按鈕變成「取消預約」。成功訊息是一行字，不是綠色橫幅。額滿的時候顯示灰色的「額滿」兩個字，不是紅色標籤。額滿是一種狀態，不是警告。

先問自己：「我的預約」頁的「取消」按錯了，會怎樣？先想一下再往下讀。

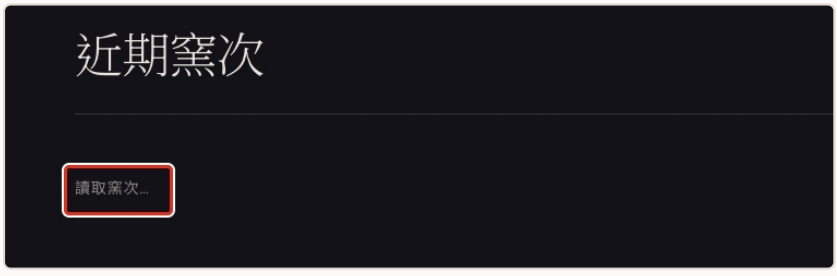
會直接刪掉，沒有回頭路。名額空出來，可能馬上被別人搶走。參考成品的取消要按兩次。第一次只把文字換成「再按一次確認取消」，4 秒內沒再按就還原。預約或取消之後，鍵盤焦點也會移到新的狀態那一行，不會掉回頁首。

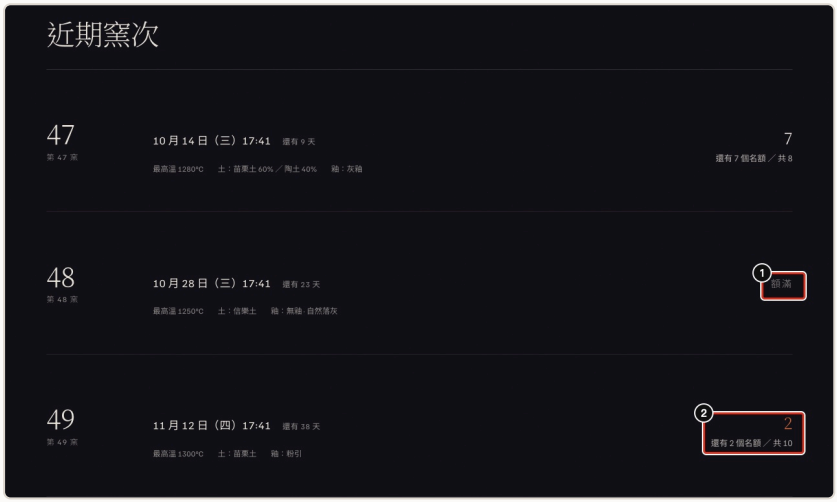
> **名詞**：**鍵盤焦點**是用 Tab 鍵操作時，目前停在哪一個按鈕或連結上。例如按完「預約這一窯」，焦點停在那個按鈕上。
> 你需要知道到什麼程度：知道畫面重畫之後焦點可能掉回頁首。要請 Claude 把焦點移到新的狀態那一行。怎麼寫不用會。

你的版本沒有這兩件事也算做對。想加的話貼這段：

> 貼給 AI
>
> 「取消」改成要按兩次：第一次按下只把文字換成「再按一次確認取消」，4 秒內沒再按就還原。不要用彈出視窗。
> 預約或取消之後畫面會重畫，請把鍵盤焦點移到新的狀態那一行，不要讓它掉回頁首。

另外三種狀態的參考畫面在下面。這是參考成品的樣子，你的版本文字可以不同。但不能出現英文錯誤原文、紅綠顏色或轉圈圈。

狀態	參考畫面
讀取中	 你要看：只有一行「讀取窯次…」，沒有轉圈圈

狀態	參考畫面
錯誤	 你要看：一句中文說明發生什麼事，沒有英文錯誤原文，也沒有紅色
額滿	 你要看：框 1 是灰色的「額滿」兩個字，沒有預約按鈕，也沒有紅色標籤。框 2 是名額剩很少的那一窯，數字用火焰橘（印成黑白看不出顏色，所以用框標出來）

🧩 它在建構流程的位置

> 登入 ⟶ 【預約與取消】 ⟶ 正式環境驗證 ⟶ 重現漏洞

💡 那段指令為什麼那樣寫

第 5 點：錯誤訊息要翻譯。

資料庫回的是 `duplicate key value violates unique constraint`。這句話對使用者毫無意義。但它對你有意義：階段 5 設的那條規則生效了。

你的工作是把它翻成人話，同時把原文留下來。

> **名詞**：console 是瀏覽器開發者工具裡的一個分頁，程式印出來的訊息都在那裡。
> 你需要知道到什麼程度：知道錯誤原文會印在那裡。出事的時候，整段複製貼給 Claude。怎麼打開可以問它。

翻譯的原則：

給誰看	應該看到什麼
使用者	「你已經預約過這一窯了」
你（開發者）	完整的原始錯誤，印在 console 裡

兩者都要有。只留人話而不印原文，你自己就少了除錯的線索。

階段 12　在正式環境上再驗一次規則

📖 讀完約 10 分鐘　🛠️ 動手約 30 分鐘

- **這一步一定要懂**：階段 8 是在替代品上測的。同一份測試要在真的 Supabase 上再跑一次。**看到 2xx 不等於操作成功**，要回頭查資料。
- **這一步可以先不懂**：Supabase 每個選單的用途、HTTP 狀態碼的細節。
- **做完後你要能用自己的話回答**：訪客碰不到預約表，是第一層擋的還是第二層？兩者差在哪？

⚖️ 動手前先想

階段 8 那一題再問一次：這個規則有沒有真的生效？

SQL Editor 也是用管理員身分執行。測試裡少了換身分那幾行，結果就和階段 8–5 註解掉時一樣。

先自己想：Claude 給你的任何一段驗權限的 SQL，你怎麼知道它換過身分？

以後拿到這類 SQL，先問這一句，再到它指出的那幾行對一次：

貼給 AI

```
請指出這段 SQL 是在哪幾行換成 anon 或 authenticated 的身分，
以及哪幾行還是用管理員身分在跑。
```

🛠️ 動手做

🧰 這一步的產出

不會多出檔案。你會在 Supabase 的 SQL Editor 裡看到七行 ✓。

步驟一：造出兩個真的使用者

用**兩個不同的 email** 在你的網站上各登入一次。先用第一個登入，**登出**，再用第二個登入。

> **只有一個信箱怎麼辦**：Gmail 支援別名。如果你的信箱是 `abc@gmail.com`，那 `abc+b@gmail.com` 會寄到同一個信箱，但 Supabase 會把它當成兩個不同的使用者。

步驟二：拿到他們的 id

Supabase 網站 → **SQL Editor** → **New query** → 貼上：

```
select id, email from auth.users order by created_at limit 5;
```

按 **Run**，下方的 **Results** 分頁至少會出現兩列（階段 10 用過別的信箱就會多一列）。用 email 認出步驟一那兩個帳號，先登入的當 A。**把那兩串長長的 id 複製下來。**

步驟三：跑測試

1. 用兩個帳號各到網站的「我的預約」，把階段 11 留下的預約全部取消。測試會自己預約 47 窯，A 已經約過的話，測試會在第 3 項中斷
2. 把 `supabase/rls-test.sql` 整份貼進 SQL Editor
3. 把裡面的 `<A>` 和 `<B>` 換成剛剛那兩串 id
4. 按 **Run**

結果會出現在下方的 **Messages** 分頁（**不是 Results 分頁**）。七行的樣子和階段 8〈執行之後應該看到〉相同。

> **測試最後會自動復原**，所以不會在正式資料裡留下任何東西。這一點要先知道，否則你會不敢在正式專案上按 Run。

步驟四：確認縱深防禦

> **名詞**：**縱深防禦**是同一件事用兩層擋，一層寫錯了還有另一層。
> 你需要知道到什麼程度：知道這一步要讓訪客在第一層就被擋下。

貼給 AI

你只需要修改：沒有。整段照貼。

```
請幫我確認：訪客（沒登入的人）碰得碰不到 reservations 這張表？

我要知道他是在第一層被擋（permission denied），
還是在第二層被擋（讀得到表但拿到 0 列）。

如果是第二層，請告訴我要怎麼改成第一層就擋下來，並說明為什麼要這樣做。
```

它如果回答「目前是第二層擋的」，請它照它說的改一次。把更新後的 `schema.sql` 重貼到 SQL Editor 執行，然後重跑一次測試。

✅ 怎麼知道做對了

- [ ] 在正式專案上七項全部 ✓
- [ ] 我說得出本機測試和正式環境測試各補什麼。本機改一次政策就能立刻驗一次。正式環境驗的是使用者真的會連到的那一套設定。
- [ ] 訪客是在**第一層**就被擋下，不是第二層
- [ ] 我知道「改完政策要重跑測試」
- [ ] 我知道看到 2xx 不代表操作真的成功了

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
測試 2、5、6 出現 ✗，B 看得到別人的預約	用管理員身分測，政策被無視了	確認測試裡有「換身分」那幾行
Messages 分頁沒東西	看錯分頁了	結果在 **Messages**，不是 Results
找不到 `<A>` 要換哪裡	階段 8–2 的第 4 點沒做	「請把測試裡的使用者 id 改成 `<A>` `<B>` 佔位符」
只有一個使用者	第二個沒登入成功	用 Gmail 別名再登入一次
跑到第 3 項就出現 `duplicate key`	A 之前已經約過 47 窯	先到網站取消，再重跑
測試 6 說 B 看得到 1 筆	B 之前有預約還沒取消	先到網站取消，再重跑
測試 7 的數字不是 7	示範資料的名額被改過（例如之後回來重跑時，階段 13、15 改過的名額還沒還原）	先重跑一次 `seed.sql`。還是不對，跟 Claude 說：「請列出 47 窯目前有幾筆預約。」

這一步失敗可以跳過嗎：不行。這是唯一能證明你的規則在真實環境有效的方法。

📖 為什麼這樣做

🎯 這一步的重點

理解**本機測試和正式環境測試各補什麼**，以及為什麼兩個都要做。

🧭 這一步在做什麼

階段 8 是在自己電腦上用替代品測。這一步是在**真的環境**測，因為替代品終究是替代品。

兩個都要做。本機改一次政策就能立刻驗一次。正式環境驗的是使用者真的會連到的那一套設定。

🧩 它在建構流程的位置

> 預約與取消 ⟶ 【正式環境驗證】 ⟶ 重現漏洞

正式環境會多發現什麼

先自己想：訪客讀預約表被擋下，你怎麼分辨是第一層還是第二層擋的？

看訊息：第一層擋下會出現 `permission denied for table reservations`。第二層擋下沒有錯誤，只是拿到 0 列。

在真的 Supabase 專案上，會發現一件本機完全測不到的事：

> Supabase 的專案預設就把 `public` 的權限給了訪客角色。
>
> 所以你以為「訪客碰不到預約那張表」，是第一層擋的。實際上他碰得到，只是政策讓他一列也拿不到，那是第二層擋的。
>
> **你以為兩層都在守，其實只有一層。**

修法是明確地「收回」權限，也就是階段 6–1 第 4 點在做的事。收回之後，訪客會在第一層就被擋下，根本走不到第二層。

這叫縱深防禦。哪天政策寫錯了，第一層還擋著。

想直接看訪客手上還有哪些權限：

貼給 AI

```
請寫一段 SQL 讓我貼到 SQL Editor：列出 anon（訪客）對 reservations 這張表還有哪些
權限。
收回之後，正確的結果應該是 0 列。
```

一個會騙人的回應碼

還有一件事。訪客嘗試刪除別人的預約，伺服器回的是 **HTTP 204**，看起來像「成功了」。

但它的意思是「這個操作跑完了，影響 0 列」。

要確認有沒有真的被擋，有兩個方法：

1. **回頭查一次資料**：看它有沒有真的變（最直觀）
2. 在請求上要求伺服器回傳「實際影響了哪幾列」，被擋下時會是空的

先自己想：A 去刪 B 的預約，畫面上沒有任何錯誤。B 那一筆還在嗎？

第 1 個方法可以請 Claude 照 `rls-test.sql` 的做法寫。拿到之後把 `<A>` `<B>` 換成步驟二的兩個 id：

貼給 AI

```
請寫一段和 rls-test.sql 同樣做法的 SQL（換身分、最後復原）：
先用管理員身分替 <B> 預約 47 窯，再換成 <A> 的身分刪除那一筆。
印出刪除影響了幾列，以及 B 的那一筆還在不在。
```

規則有效的話，影響 0 列，B 那一筆還在。

> 看到 2xx 不等於「他成功改到你的資料」。判斷有沒有被擋，要看資料本身，不要看回應碼。

💡 那段指令為什麼那樣寫

〈🛠️ 動手做〉裡只有步驟四要貼 Prompt。步驟一在你的網站上登入，步驟二、三在 SQL Editor 操作。

> 其他三段 Prompt 在〈⚖️ 動手前先想〉和〈📖 為什麼這樣做〉，是讓你自己核對用的。

🧭 Checkpoint 3（做完階段 12）

網站現在會動了：看得到、登入得了、預約得了，測試也全過。

1. 本機測試和正式環境測試，各補了什麼？
2. 訪客碰不到預約那張表，是第一層擋的還是第二層？為什麼這個差別重要？
3. 看起來都正常了。**你現在敢部署嗎？為什麼？**

> 第 3 題如果你答「敢」，階段 13 和 16 會示範它怎麼出事。 **兩個人同時按下同一個按鈕的情況，你還沒有測過。**

階段 13　重現問題：兩個人搶最後一個名額

📖 讀完約 6 分鐘　🛠️ 動手約 25 分鐘

- **這一步一定要懂**：「檢查名額」和「寫入預約」是兩個分開的動作，中間有時間差。兩個瀏覽器互相不知道對方存在。
- **這一步可以先不懂**：怎麼精準地同時送出兩個請求。對不準就多試幾次。
- **做完後你要能用自己的話回答**：為什麼這件事前端不可能解決？

⚖️ 動手前先想

這個版本可不可以對外？

到這一步為止，網站是會動的，看起來也沒問題。**但你即將親眼看到它會出錯。**

網站會動，不代表可以交給別人用。中間差的就是階段 13 到 16 這四步。

🛠️ 動手做

🧰 這一步的產出

不會多出檔案。你會得到一張「剩餘名額 −1」的畫面，以及一個親身經驗。

步驟一：把某一窯的名額調到只剩 1

> **你只需要修改**：只有「47 窯」換成你資料裡的窯次編號。最後那句「這個改動要怎麼還原」照貼。

Claude 手上只有可以公開的那把金鑰。它改不了窯次的名額（階段 6 把這個權限收回了）。所以這一段是請它寫 SQL，由你貼到 SQL Editor 執行。

```
我要重現一個超額預約的問題，需要一個「只剩 1 個名額」的窯次。

請寫一段 SQL 給我，我貼到 Supabase 的 SQL Editor 執行：
把 47 窯的名額改成「目前已預約數 + 1」，讓它剛好只剩一個位子。

先告訴我這段 SQL 會做什麼，以及這個改動要怎麼還原。
```

拿到 SQL 之後貼到 SQL Editor 按 **Run**。再執行下面這一句確認，`seats_left` 應該是 1：

```
select kiln_no, seats_left from firings_with_seats() where kiln_no = 47;
```

步驟二：兩個瀏覽器同時按

1. 開**兩個不同的瀏覽器**（例如 Chrome 和 Edge，Mac 也可以用 Safari）。也可以用一個正常視窗加一個無痕視窗
2. 分別用你的兩個帳號登入。**登入信裡的連結不要直接點**，它會開在你的預設瀏覽器，另一個視窗還是沒登入。把連結複製起來，貼到那個視窗的網址列
3. 兩邊都進到那一窯的頁面，**兩邊都會顯示「還有 1 個名額」**
4. **同時**按下預約

> **同時按不容易對準怎麼辦**：兩隻手各按一個視窗，數 3、2、1 一起按。沒對準的話多試幾次。也可以請 Claude 幫你寫一個同時送出兩個請求的小測試。

步驟三：看結果

重新整理任一個頁面。兩個帳號都預約成功了，畫面上這一窯顯示「額滿」。

畫面不會顯示負數，實際的數字要到資料庫看。到 SQL Editor 執行下面這一句。函式名稱以你的 `schema.sql` 為準，不一樣就請 Claude 換掉：

```
select kiln_no, seats_left from firings_with_seats() where kiln_no = 47;
```

剩餘名額會是 −1。

✅ 怎麼知道做對了

- [ ] **我親眼看到剩餘名額變成 −1**（在 SQL Editor 查出來的）
- [ ] 我解釋得出為什麼前端解決不了這件事
- [ ] 我說得出這件事和階段 3 第 2 題的關係
- [ ] 我記得自己在階段 3 是怎麼回答的

🆘 如果沒有成功

你可能會遇到	通常是因為	怎麼辦
一直只有一筆成功	兩次按的時間差太大	多試幾次。或請 Claude 寫一個同時送出兩個請求的小測試
第二個瀏覽器登入不了	用了同一個帳號	要用兩個不同的帳號
名額調不到剩 1	已有的預約數不對	「請告訴我 47 窯目前有幾筆預約，我要把名額調成剛好剩 1」

這一步失敗可以跳過嗎：不建議。 這是階段 15 的驗收基準，跳過的話你沒有東西可以對照。真的重現不出來，至少要能說出它為什麼會發生。

📖 為什麼這樣做

🎯 這一步的重點

親手重現一個漏洞，理解為什麼「前端的檢查不算數」。

🧭 這一步在做什麼

這個網站有一個已知的問題：兩個人同時搶最後一個名額，兩個都會成功。

> **名詞：超額預約**是名額只剩一個，卻寫進去兩筆預約。它的正式名稱叫 race condition。
> 你需要知道到什麼程度：知道它發生的條件是「查名額」和「寫預約」分成兩步。正式名稱不用記。

這是故意留到現在的。 你要親手重現它一次。

你會看到的結果：在 SQL Editor 查 47 窯的剩餘名額，結果是 **−1**。兩個瀏覽器在按下去之前，畫面上都顯示「還有 1 個」。它們互相不知道對方存在，所以兩邊都判斷自己這一筆是合法的。

🧩 它在建構流程的位置

```
正式環境驗證 ──▶ 【重現漏洞】 ──▶ 修掉 ──▶ 再驗一次 ──▶ 堵住繞過 ──▶ 部署
```

不要跳過這一步直接去修。 先看到問題再修，你才會相信它真的存在，也才知道修好之後要怎麼驗。

為什麼會這樣

因為「檢查名額」和「寫入預約」是**兩個分開的動作**，中間有時間差：

```
A 的瀏覽器：查詢 → 還有 1 個 →（此時 B 也查到 1 個）→ 寫入 ✓
B 的瀏覽器：查詢 → 還有 1 個 →                        → 寫入 ✓
```

前端不可能解決這件事，因為兩個瀏覽器互相不知道對方存在。

> 這就是階段 3 第 2 題的答案，只是現在你親眼看到了。 **你當時是怎麼回答的？**

💡 那段指令為什麼那樣寫

這一步幾乎不用下指令，是動手實驗。唯一要 AI 幫忙的是**把名額調到只剩 1**。

階段 14　把「檢查」和「寫入」合成一個動作

📖 讀完約 8 分鐘　🛠️ 動手約 35 分鐘

- **這一步一定要懂**：「要嘛全部成功、要嘛全部不算」，而且**只有資料庫做得到**。因為只有它同時看得到所有人的請求。
- **這一步可以先不懂**：交易、鎖、資料庫函式怎麼寫。你只看中文註解與白話解釋。
- **做完後你要能用自己的話回答**：修好之後，搶輸的那個人會看到什麼？那句話是誰決定的？

🛠️ 動手做

🧰 這一步的產出

```
supabase/schema.sql        ← 新增一個「預約」的資料庫函式
src/pages/firing.astro     ← 改成呼叫那個函式，不再自己先查再寫
```

畫面上看起來沒有變化。變的是資料庫那一層，你要靠階段 15 的實驗才看得到。

📋 貼給 AI（14–1：先解釋）

貼給 AI

你只需要修改：沒有。整段照貼。**尤其「先不要寫程式碼」和第 4 點（搶輸的人會看到什麼）。**

```
我的預約系統有一個問題：兩個人同時搶最後一個名額，兩筆都會寫進去，
剩餘名額變成 -1。

先不要寫程式碼。請用沒有程式背景的人聽得懂的方式，說明：

1. 為什麼前端不可能解決這件事
2. 資料庫要保證什麼，才能解決
3. 你打算怎麼做？這個做法怎麼擋住兩個同時發生的請求
4. 修好之後，搶輸的那個人會看到什麼訊息

我確認之後你再實作。
```

📋 貼給 AI（14-2：實作）

貼給 AI

你只需要修改：沒有。整段照貼。

可以，請實作。要求：

1. 在資料庫裡寫一個「預約」的函式，把「檢查名額」和「寫入預約」包成一個不可分割的動作
2. 名額不足時要明確地失敗，並回傳一個前端看得懂的代號
3. 前端改成呼叫這個函式，不要再自己先查再寫
4. 前端要把兩種失敗分別翻成人話：
   - 已經預約過這一窯
   - 名額剛好被搶完
5. 這份 SQL 一樣要能重複執行不出錯
6. 預約的人是誰，由資料庫自己判斷（`auth.uid()`），不要讓前端傳進來

請用中文註解說明：為什麼這樣就不會有兩個人同時成功。

你自己做：把更新後的 schema.sql 再執行一次

把更新後的 `schema.sql` 再貼到 Supabase 的 SQL Editor 執行一次。和階段 6 一樣，NOTICE 或 skipping 是正常的，只有 ERROR 才是錯。

✅ 怎麼知道做對了

- [ ] Claude 用白話解釋過為什麼前端做不到，而且我聽懂了
- [ ] `schema.sql` 重複執行不出錯
- [ ] 前端不再是「先查再寫」，而是呼叫一個函式
- [ ] 名額不足時，使用者看到的是人話，不是英文錯誤
- [ ] 中文註解說得出「為什麼這樣就不會有兩個人同時成功」

第三項不用讀程式碼。問 Claude：「現在前端是先查名額再寫入，還是直接呼叫那個函式？請指出是哪一行。」

但真正的驗收在下一步。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
一般的預約也失敗了	函式的條件寫太嚴	「現在連正常預約都失敗，請檢查名額判斷的條件」
還是會超額	檢查和寫入還是分開的	「還是會超額，請確認檢查和寫入是在同一個不可分割的動作裡」
錯誤訊息還是英文	第 4 點沒做	「請把兩種失敗分別翻成人話」
SQL 執行出錯	語法問題	整段錯誤貼給 Claude

這一步失敗可以跳過嗎：可以。但那樣你的作品就停在 Learning Prototype，而且**不應該部署**。要跳過的話，在你的筆記上明確寫下這個已知問題。

📖 為什麼這樣做

🎯 這一步的重點

理解「要嘛全部成功、要嘛全部不算」是什麼意思，以及為什麼只有資料庫做得到。

🧭 這一步在做什麼

修掉階段 13 那個問題。

正確的解法是把「檢查名額」和「寫入預約」合成一個不可分割的動作，交給資料庫做。

> **名詞：交易（transaction）** 是資料庫的一種保證。這幾件事要嘛全部成功，要嘛全部不算。你需要知道到什麼程度：知道「查名額」和「寫預約」要包在同一個交易裡。轉帳是最典型的例子，錢從 A 扣掉、加到 B，不能只做一半。

🧩 它在建構流程的位置

重現漏洞 ⟶ 【修掉】 ⟶ 再驗一次 ⟶ 堵住繞過 ⟶ 部署

為什麼前端做不到

不管你在前端寫得多小心，順序永遠是先問資料庫還有幾個，再叫資料庫寫入。中間一定有時間差。

只有資料庫本身能解決，因為**只有它同時看得到所有人的請求**。

先自己想：兩個預約同時到了資料庫，要怎麼做，第二個人數名額時才會數到第一個人？

參考成品的做法是排隊：`reserve()` 先鎖住那一窯，第二個請求要等第一個做完，才往下數名額。做對了，階段 15 查到的剩餘名額是 0，不是 −1。

不用讀程式碼，請 Claude 指出來：

貼給 AI

> 請指出預約函式裡是哪一行讓同一窯的預約一次只處理一個，
>
> 以及第二個人等完之後，數到的人數包不包含第一個人。

💡 那段指令為什麼那樣寫

14–1 和 14–2 分成兩段是刻意的：先要它用白話解釋，你確認之後才實作。

14–1 那四個要解釋的問題：

1. 為什麼前端不可能解決
2. 資料庫要保證什麼
3. 這個解法怎麼擋住兩個同時發生的請求
4. 失敗的那個人會看到什麼

第 4 點最容易被忽略。 修好之後一定有一個人會搶輸。他看到的是一句人話還是一段英文，決定這次修改做完了沒。

階段 15　用同一個方法再驗一次

📖 讀完約 4 分鐘　🛠️ 動手約 20 分鐘

- **這一步一定要懂**：方法要和階段 13 完全一樣，但窯次要換**另外一窯**。階段 13 那一窯你兩個帳號都約過了，不能再約第二次。
- **這一步可以先不懂**：階段 14 那個資料庫函式內部怎麼運作。這一步是動手實驗，不寫程式碼。
- **做完後你要能用自己的話回答**：換一個測法而它通過了，為什麼證明不了原本的問題修好了？

⚖️ 動手前先想

階段 13 那一題再問一次：這個版本可不可以對外？

做完這一步，你會有一半的依據：已知的問題修掉了，而且用原本的方法驗過了。

另一半在階段 16。階段 13 和這一步的實驗都是從網頁按按鈕。這一步走的是階段 14 建的函式。不經過網頁的人走哪一條路，還沒有人檢查過。

🛠️ 動手做

🧰 這一步的產出

不會多出檔案。你會得到一個結果：**只有一筆成功**。

步驟

和階段 13 完全一樣：

1. 請 Claude 寫 SQL，把**另外一窯**的名額調到只剩 1。你貼到 SQL Editor 執行。階段 13 那一窯你兩個帳號都約過了，不能再約第二次。「48 窯」換成你資料裡的另一窯：

貼給 AI

```
請寫一段 SQL 給我，我貼到 Supabase 的 SQL Editor 執行：
把 48 窯的名額改成「目前已預約數 + 1」，讓它剛好只剩一個位子。
先告訴我這段 SQL 會做什麼，以及這個改動要怎麼還原。
```

2. 兩個瀏覽器、兩個帳號，都進到那一窯的頁面
3. **同時**按下預約
4. 重新整理

✅ 怎麼知道做對了

- [ ] **只有一筆成功**
- [ ] 用階段 13 那一句 SQL 查（`kiln_no` 換成這一窯，例如 `select kiln_no, seats_left from firings_with_seats() where kiln_no = 48;`），剩餘名額是 **0**，不是 −1
- [ ] 搶輸的那個人看到一句人話，例如「這個場次剛剛已額滿，請重新整理查看其他場次」
- [ ] 搶輸的那個人**沒有**看到英文錯誤或空白畫面
- [ ] 我用的是和階段 13 完全一樣的方法

全部打勾之後，還差一步。階段 16 要堵住不經過網頁的那條路。做完才是 Functional Prototype，才可以部署。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
還是兩筆都成功	修改沒生效，或 SQL 沒重新執行	確認更新後的 `schema.sql` 有貼到 Supabase 執行過
兩個都失敗了	條件寫太嚴	「現在兩個都失敗了，請檢查名額判斷的條件」
搶輸的看到英文	翻譯沒做完整	「名額不足的錯誤還是英文，請翻成人話」
搶輸的看到空白	沒有處理失敗的情況	「搶輸的時候畫面是空的，請顯示一句說明」

這一步失敗可以跳過嗎：不行。 沒有用階段 13 的方法再驗一次，你不知道 −1 還會不會出現。

📖 為什麼這樣做

🎯 這一步的重點

修完一定要用原本的方法再驗一次。「沒有錯誤訊息」不等於「修好了」。

🧭 這一步在做什麼

用階段 13 一模一樣的方法，再做一次同樣的實驗。

方法必須一模一樣，否則你證明不了什麼。換一個測法而它通過了，你只知道新的測法會過，不知道舊的問題修好了。

🧩 它在建構流程的位置

```
修掉 ──► 【再驗一次】 ──► 堵住繞過 ──► 部署
```

這一步通過了，才進到階段 16。

💡 那段指令為什麼那樣寫

幾乎不用下指令。這是動手實驗。

階段 16　堵住繞過網頁的那條路

📖 讀完約 6 分鐘　🛠️ 動手約 25 分鐘

- **這一步一定要懂**：階段 14 的函式只管「透過函式」的預約。使用者手上還有直接寫入預約表的權限，那一條路沒有檢查名額。
- **這一步可以先不懂**：收回權限的 SQL 語法、函式用誰的身分執行。
- **做完後你要能用自己的話回答**：階段 15 的實驗為什麼抓不到這個問題？

⚖️ 動手前先想

階段 3 那一題再問一次：有人繞過網頁直接送請求，還擋不擋得住？

階段 14 的函式擋得住兩個人同時按。那不經過函式、直接寫入預約表的路呢？

🛠️ 動手做

🧰 這一步的產出

```
supabase/schema.sql       ← 收回直接新增預約的權限
supabase/rls-test.sql     ← 測試 3、4 改成透過函式預約，測試 5 改成驗證「繞過會被擋下」
```

畫面上沒有變化。

📋 貼給 AI（16–1：重現繞過）

貼給 AI

你只需要修改：只有「50 窯」換成你資料裡的一窯。**最後那句「不要留下任何資料」照貼。** `<A>` `<B>` 先照抄，拿到 SQL 之後再換。

```
我要確認：有人不經過網頁、也不經過預約函式，直接寫入預約表，擋不擋得住。

請寫一段 SQL 給我，我貼到 Supabase 的 SQL Editor 執行：
1. 先用管理員身分替 <A> 在 50 窯建一筆預約，再把 50 窯的名額調成目前的預約數，
   讓它剛好額滿（名額不能設成 0）
2. 用 rls-test.sql 一樣的方法，換成 <B> 的身分（登入的使用者，不是管理員）
3. 不呼叫預約函式，直接對 reservations 新增一筆 50 窯的預約
4. 印出 50 窯的剩餘名額

整段包在一個交易裡，最後復原，不要留下任何資料。
```

拿到 SQL 之後，把 `<A>` `<B>` 換成階段 12 拿到的兩個使用者 id。再貼到 SQL Editor 執行。

你應該看到：剩餘名額是 **−1**。函式擋得住兩個人同時按，擋不住不經過它的人。

📋 貼給 AI（16–2：堵住）

貼給 AI

你只需要修改：沒有。整段照貼。**第 2 點不要刪**，它在檢查階段 14 的函式能不能替別人預約。

```
剛剛的實驗證明，登入的使用者可以繞過預約函式，直接寫入預約表。請修掉：

1. 收回登入使用者直接新增預約的權限，讓預約只能透過預約函式
2. 確認預約函式裡的使用者身分是資料庫自己判斷的（auth.uid()），
   不是由前端傳進來的。如果是前端傳的，一起改掉
3. 「只能新增自己的預約」那條政策留著，並用中文註解說明為什麼留
4. 把 rls-test.sql 的測試 3、4 改成透過預約函式預約；
   測試 5 改成「A 繞過函式直接寫入，還冒用 B 的身分」，預期被第一層擋下
```

5. `schema.sql` 一樣要能重複執行。已經執行過舊版的專案重跑這份，
   也要把舊的權限收回來

改完用白話告訴我改了什麼，以及為什麼這樣就擋得住。

你自己做（16-3：重跑三次）

1. 把更新後的 `schema.sql` 貼到 SQL Editor 執行
2. 再跑一次 16-1 那段 SQL。這次應該在新增那一步就停下來，出現 `permission denied for table reservations`
3. 用兩個帳號到網站的「我的預約」把預約全部取消。再把 `seed.sql` 貼到 SQL Editor 重跑一次，把階段 13、15 改過的名額還原。最後把更新後的 `rls-test.sql` 換好 `<A>` `<B>` 執行
4. 到網站上用一個帳號預約一窯，確認正常的預約還是成功

✅ 怎麼知道做對了

- [ ] 16-1 第一次跑，剩餘名額是 −1
- [ ] 改完再跑 16-1，出現 `permission denied for table reservations`
- [ ] `rls-test.sql` 七項全部 ✓，測試 5 顯示繞過函式直接寫入被擋下
- [ ] 網站上正常預約還是成功
- [ ] 我說得出階段 15 的實驗為什麼抓不到這個問題

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
網站上的預約全部失敗	前端還在直接寫入預約表，沒有呼叫函式	「網站上預約失敗了，請確認前端是呼叫預約函式，不是直接寫入 reservations」
16-1 第二次跑還是 −1	舊的權限沒有收回	「重跑 schema.sql 之後，登入的使用者還有新增預約的權限，請加上明確收回的那一行」
測試 3 出現 `permission denied`	測試 3 還在直接寫入	「測試 3 還在直接寫入預約表，請改成呼叫預約函式」
測試 3 出現 `duplicate key` 或 `already_reserved`	預約沒有先取消	先到網站取消，再重跑

你可能會看到	通常是因為	怎麼辦
測試 3 出現 `firing_full`，或測試 7 說還剩 0 個	階段 13、15 改過的名額沒有還原	重跑一次 `seed.sql`

這一步失敗可以跳過嗎：不建議。跳過的話，網站看起來一切正常，但任何登入的人都能繞過網頁把名額灌爆。

📖 為什麼這樣做

🎯 這一步的重點

修好一個漏洞之後，再問一次：除了這個函式，還有沒有別的方法能把預約寫進預約表。

🧭 這一步在做什麼

階段 14 把「檢查名額」和「寫入預約」合成一個函式。階段 15 驗證了兩個人同時按，只有一筆成功。

但這兩個實驗都是從網頁按按鈕，走的都是函式。使用者手上那把可以公開的金鑰，還能直接寫入預約表。這條路從頭到尾沒有檢查過名額。

先自己想：登入的使用者對 `reservations` 手上有哪幾種權限？每一種會不會經過預約函式？

16–1 會量給你看：直接寫入之後，剩餘名額變成 −1。以後每修好一個漏洞，都可以這樣問一次：

貼給 AI

```
請列出登入的使用者（authenticated）對每一張表有哪些權限（讀取、新增、修改、刪除），
以及它能執行哪些函式。每一項說明它會不會經過名額檢查。
```

收回權限之後

這次要擋的不是前端的檢查，是登入使用者手上那個直接新增預約的權限。

權限分兩層。階段 6 收回的是訪客碰預約表的權限。這一步收回的是登入使用者新增預約的權限。收回之後，直接寫入在第一層就被擋下，走不到第二層的政策。

「只能新增自己的預約」那條政策因此用不到了，但還是留著。哪天有人又把新增的權限給回去，它還擋著冒用別人身分的寫入。

🧩 它在建構流程的位置

> 修掉 ──▶ 再驗一次 ──▶ 【堵住繞過】 ──▶ 部署

💡 那段指令為什麼那樣寫

16–1 用的是和 `rls-test.sql` 同一套做法：換身分、包在交易裡、最後復原。不換身分的話，SQL Editor 是用管理員身分執行，什麼都擋不住。這個實驗就沒有意義。

16–2 第 2 點在檢查一個常見的寫法：函式接受前端傳來的使用者 id。那樣的話，收回權限之後，任何人還是可以透過函式替別人預約。

怎麼看出來：問 Claude「前端呼叫預約函式時傳了哪幾個值？」參考成品只傳窯次的 id（`p_firing_id`）。多傳了使用者 id，就是這個寫法。

階段 17　推上 GitHub 並部署成公開網址

📖 讀完約 8 分鐘　🛠️ 動手約 40 分鐘

- **這一步一定要懂**：有兩個一定會忘記的設定。環境變數改了要重新部署，正式網址要加進 Redirect URLs。上線關卡擋的是推出去就收不回來的兩件事。一件是前端程式碼裡的金鑰，一件是被加進版本控制的 `.env`。
- **這一步可以先不懂**：Vercel 設定畫面每個欄位的意思、打包怎麼跑。
- **做完後你要能用自己的話回答**：改了環境變數，為什麼一定要重新部署一次才生效？

⚖️ 動手前先想

回到操作卡 2 的三層原則。建立 GitHub 專案、第一次推上去、建立雲端資源，屬於第三層。**這三件 AI 都要先說明，你點頭才能做。**

🛠️ 動手做

🧰 這一步的產出

- `scripts/predeploy-check.mjs`：上線關卡，推上 GitHub 之前自動跑
- 一個 GitHub 專案
- 一個可以打開的公開網址
- 一份上線前的安全檢查結果

📋 貼給 AI（17-1：先裝一道上線關卡）

17-2 會請 Claude 逐檔搜尋金鑰。那是 Claude 逐檔讀過去，可能漏掉。這一段把其中兩件推出去就收不回來的事，做成推送前一定會跑的檢查。沒通過，`git push` 根本不會執行。

階段 4 搬過來的自動檢查，是在 Claude **改完檔案之後**跑。這一道是在 Claude **執行指令之前**跑，所以擋得住還沒發生的事。

貼給 AI

你只需要修改：沒有。整段照貼。**「註解裡提到不算」那一句不要刪**，`src/lib/supabase.js` 的註解本來就有提到這個詞。

```
建立 scripts/predeploy-check.mjs，作為推上 GitHub 之前的關卡。

行為規格：
- 從標準輸入讀 Claude Code 傳來的資料（JSON 格式），取出它準備執行的指令
- 只有指令裡有 git push，或 gh repo create 加上 --push 時才檢查，其他指令直接放行
- 檢查兩件事：
  1. src/ 和 public/ 裡有沒有出現 service_role 或 sb_secret_。註解裡提到不算
  2. .env 有沒有被加進版本控制（用 git ls-files 確認）
- 任何一件沒過，就印出原因和該怎麼修，結束代碼是 2
- 加一個 --all 參數，可以手動檢查一次，通過時印出「全部通過」

錯誤訊息用繁體中文，說明為什麼這兩件事推出去就收不回來。

然後在 .claude/settings.json 裡加一個「執行指令之前」的檢查（PreToolUse），
對象是 Bash，指令是：
  node "$CLAUDE_PROJECT_DIR/scripts/predeploy-check.mjs"
原本的兩個改檔後檢查不要動。

最後故意在 src/ 建一個測試檔，內容寫 const k = 'sb_secret_test'，
然後執行 git push --dry-run。我要看關卡把它擋下來。
看完把測試檔刪掉，再跑一次 node scripts/predeploy-check.mjs --all 給我看結果。

如果關卡沒有擋下、git push 真的跑了，不要自己 git init 或設定遠端，
停下來告訴我。
```

測試用 `git push --dry-run`，是因為它就算真的執行了，也只是模擬，不會推出任何東西。

📋 貼給 AI（17-2：上線前檢查，先不要動手）

> 還沒接上 GitHub 的話，先照〈開始之前〉的〈接上 GitHub〉做一次。

貼給 AI

你只需要修改：只有 `〈…〉` 那個專案名稱。**中間那三項確認（`.env`、`service_role`、錯誤訊息）與最後那一句都不能省。**

上線前檢查。請列出這個專案目前的外部依賴風險，

每一項給我「風險是什麼」和「掛掉時畫面會變成什麼樣」：

1. Supabase（資料庫與登入）
2. 字體來源
3. 免費方案的限制

另外請幫我確認三件事：

- .env 有沒有不小心被加進版本控制
- 程式碼裡有沒有任何地方出現 service_role 或 sb_secret
- 有沒有把錯誤的原始訊息直接顯示給使用者

第二項請特別仔細檢查，逐檔搜尋。

檢查完之後，我要你幫我做這三件事：

- 把所有改動做一次 commit
- 建一個名叫 **〈kiln〉** 的 GitHub 專案，設成 private（Vercel 讀得到私人專案）
- 把程式碼推上去

但先不要動手。先把你準備執行的每一個指令列出來給我看，

每一行說明它會做什麼、會不會產生我收不回來的東西，我確認之後你再做。

📋 貼給 AI（17-3：確認之後，讓它執行）

看它列出來的指令，**特別確認 `.env` 沒有被加進去**。`.env` 進了 GitHub，那把金鑰就等於公開。

確認完再回它：

貼給 AI

你只需要修改：沒有。整段照貼。

```
可以，照你剛剛列的做。
推完之後告訴我 GitHub 專案的網址，
並且再確認一次 .env 沒有被推上去。
```

它建立專案的時候，Claude Code 會跳出一個允許視窗。 那就是操作卡 2 第三層實際運作的樣子：對外的動作，工具會停下來等你點頭。

你自己做（17–4：把 Vercel 接上，然後補環境變數）

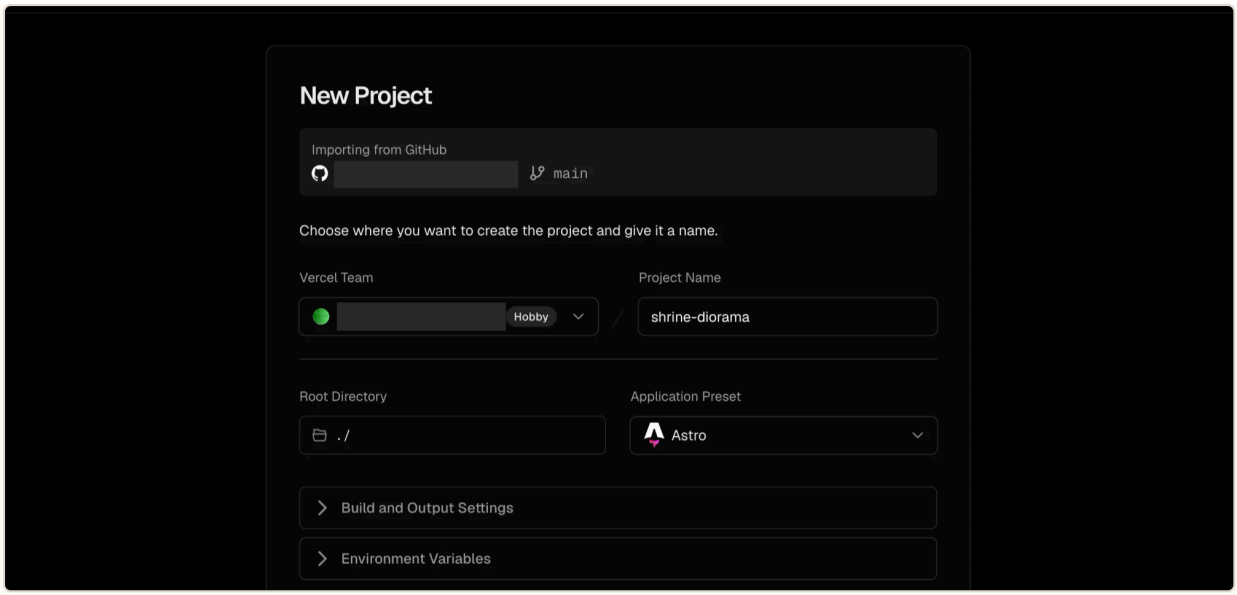
這一步只做一次。接上之後，你以後每次推上 GitHub，Vercel 都會自己重新部署。

1. 到 vercel.com，用 GitHub 帳號登入
2. 選 **Add New Project**，找到你剛剛建的 `kiln`，把它匯入
3. Vercel 會自己認出這是 Astro 專案，設定不用改
4. 設定頁最下面按 **Deploy**，跑第一次部署。上面如果有 Optional Integrations 的 Supabase **Add**，不要按：它會另外建一個新的 Supabase，不是你在階段 6 建的那個。**這一次會做出一個「尚未連上資料庫」的網站**，因為金鑰還沒設
5. 到專案的 **Environment Variables**，加入兩個值：
   - `PUBLIC_SUPABASE_URL`
   - `PUBLIC_SUPABASE_ANON_KEY`
6. **加完之後要重新部署一次**，這兩個值才會生效

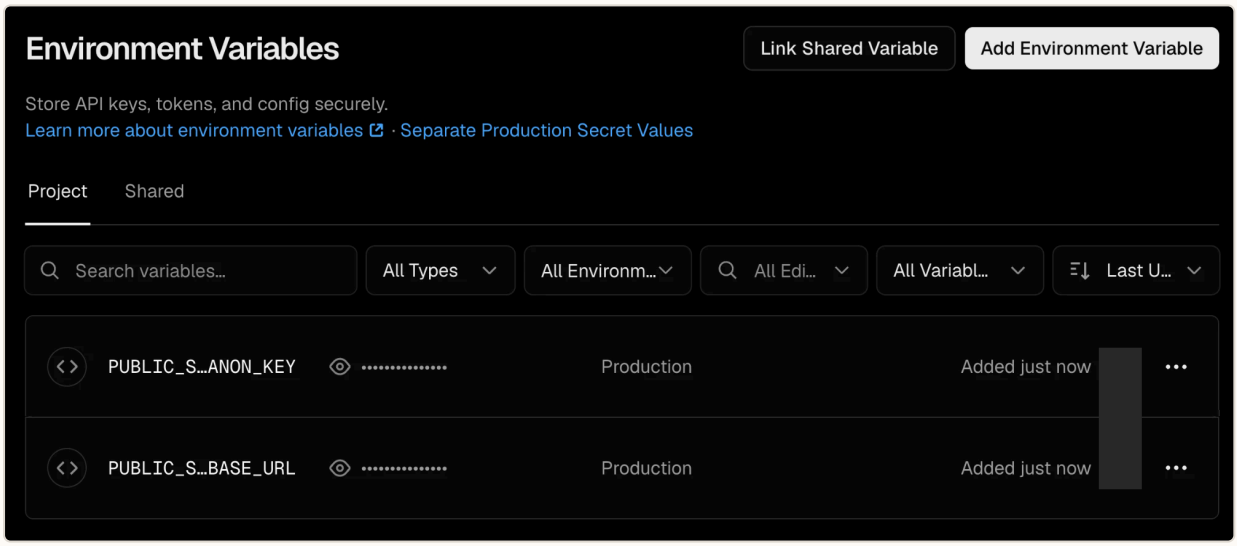
重新部署不用找按鈕。推一個新的 commit 上 GitHub，Vercel 就會自己重新部署一次：

貼給 AI

```
請推一個空的 commit 上 GitHub，讓 Vercel 重新部署。推之前先把要執行的指令列給我看。
```



第 2、3 步：匯入之後的設定頁。Application Preset 已經自己選好 **Astro**，這就是第 3 步說的「自己認出來」。畫面取自另一個練習專案，GitHub 專案名稱與 Vercel Team 已遮，Project Name 和你的不同。



第 5 步加完的樣子。畫面取自另一個練習專案。名稱太長被截成 `PUBLIC_S...ANON_KEY` 和 `PUBLIC_S...BASE_URL`，值用圓點隱藏。

第 6 步就是這一階段「一定要懂」的那件事。`PUBLIC_` 開頭的設定是在打包的時候被寫進檔案裡的，所以光是存檔沒有用，一定要讓它重跑一次打包。

> **畫面和上面寫的不一樣**：這類服務的介面常常改版。把你看到的畫面截圖貼給 Claude。跟它說「我要把這個 GitHub 專案接到 Vercel 上，還要設兩個環境變數，現在該點哪裡」。讓它帶你走。

你自己做（17-5：補上 Supabase 的設定）

回到 Supabase → **Authentication** → **URL Configuration** → 在 Redirect URLs 加上你的正式網址。

✅ 怎麼知道做對了

- [ ] 正式網址打得開，看得到五個窯次
- [ ] 在正式網址上用 email 登入成功
- [ ] 在正式網址上預約成功
- [ ] 17-1 的測試檔被上線關卡擋下來，刪掉之後 `--all` 印出「全部通過」
- [ ] `.env` 沒有被推上 GitHub
- [ ] 在正式網址後面打一個不存在的路徑（例如 `/abc`）。看到的是這個網站自己的 404 頁，不是 Vercel 的預設畫面
- [ ] 我說得出「改了環境變數為什麼要重新部署」

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
上線後登入壞掉	正式網址沒加進 Redirect URLs	補上（17-5）
改了金鑰但網站還連到舊的	`PUBLIC_` 開頭的設定是打包時寫死的	在 Vercel 重新部署一次
部署失敗	打包出錯	把 Vercel 的完整錯誤訊息貼給 Claude
17-1 的測試檔沒有被擋	對話開始時設定還沒加進去，沒被讀到	在對話框輸入一次 `/hooks`，或開一個新對話再試
推送被擋，訊息說 `.env` 被加進版本控制	忽略清單沒設好	照訊息做：`git rm --cached .env`，確認 `.gitignore` 有 `.env`，重新 commit
網站打開說「尚未連上資料庫」	環境變數沒設或設錯	檢查 Vercel 的 Environment Variables
打錯網址看到 Vercel 的預設 404	專案裡沒有 `src/pages/404.astro`	「請新增 404 頁，沿用 Base.astro 的版面，一行說明加一個回場次列表的連結」
專案打不開、一直轉圈	免費專案閒置一週被暫停	到 Supabase 的 dashboard 按恢復

你可能會看到	通常是因為	怎麼辦
`.env` 不小心推上去了	忽略清單沒設好	**立刻到 Supabase 重新產生金鑰**，然後把檔案從版本紀錄裡移除

最後一項如果發生了，先換金鑰再處理其他事。

這一步失敗可以跳過嗎：可以。網站在你電腦上跑得起來，這個案例的學習目標就達成了。

📖 為什麼這樣做

🎯 這一步的重點

處理**兩個一定會忘記的設定**，並且分辨哪些動作要 AI 先說明、你點頭才能做。

🧭 這一步在做什麼

> **名詞**：**部署**是把做好的檔案放到網路上，讓別人用網址打得開。
> 你需要知道到什麼程度：知道這個網站不需要伺服器，部署就是把檔案交給 Vercel。改一次環境變數就要重做一次。

會出錯的通常不是部署本身，是環境設定。

🧩 它在建構流程的位置

流程的終點。放在最後，是因為前面十六步都做完了，才有資格對外。

兩個一定會忘記的設定

忘記之一：環境變數沒有跟著上去

你電腦裡的 `.env` **不會**被上傳。它在忽略清單裡，這是對的。所以要在部署服務上再設一次。

> **而且要注意**：`PUBLIC_` 開頭的設定是在**打包的時候**被寫進檔案裡的。所以改了環境變數之後，**一定要重新部署一次**，改的東西才會生效。
>
> 症狀：「我明明改了金鑰，網站還是連到舊的」，就是這個原因。

忘記之二：Supabase 不認得你的正式網址

階段 10 你只加了 `http://localhost:4321`。

上線之後網址變成 `https://你的專案.vercel.app`。Supabase 沒登記過這個網址，所以**登入信裡的連結會被擋掉**。

> 上線之後登入壞掉，先查這兩項，再查別的。

💡 那段指令為什麼那樣寫

上線前的檢查裡，`service_role` 那一項不能跳過。

可以公開的那把金鑰外流沒關係。另一把外流的話，任何人都可以讀走、改掉、刪光所有資料。而且**你寫的規則完全擋不住它**。

17–1 的關卡會在每次推送前自動搜尋這個字串。17–2 再請 Claude 逐檔看一次。因為關卡只認得固定的字串，換一個寫法它就認不出來。

完成清單

- [ ] 我註冊了 Supabase 帳號，並且建立了一個專案
- [ ] 十七個階段的 ✅ 驗收都通過了
- [ ] `schema.sql` 重複執行都不會出錯
- [ ] 我在正式環境上跑過七項測試，全部 ✓
- [ ] 我親手重現過超額預約，也親手驗證過它被修好了
- [ ] 我親手重現過「不經過函式直接寫入」，也親手驗證過它被擋下
- [ ] 上線關卡 `node scripts/predeploy-check.mjs --all` 全部通過
- [ ] `.env` 沒有進版本控制
- [ ] 我知道目前的成果是 Functional Prototype，以及還缺什麼
- [ ] 我可以用自己的話說出這個案例的核心：前端的檢查是體驗，資料庫的規則才是保證

離開這個範例，我會不會自己做？

回顧

#	問題	對應階段
1	什麼情況下**不**需要資料庫？舉一個例子	2
2	兩把金鑰哪一把可以公開？為什麼另一把不行？	3
3	「能不能碰這張表」和「能碰哪幾列」分別是哪一層在管？	6
4	開了 RLS 卻一條政策都沒寫，會發生什麼事？	6
5	為什麼「還剩幾個位子」不能在前端算？	6
6	測試前為什麼一定要換身分？不換會怎樣？	8、12
7	「額滿就不顯示按鈕」能不能防止超額？為什麼？	3、11、13
8	上線後登入壞掉，最可能是哪兩個設定沒做？	17
9	你不會讀程式碼，那你怎麼確認 AI 寫的東西是對的？	整個案例

遷移

#	問題
10	如果今天做的是活動報名系統，哪些規則不能只寫在前端？
11	你要幫一個社團做「只有成員看得到的相簿」。你會怎麼設計那兩層權限？
12	客戶說「這個功能加個檢查就好了」。你會問他哪三個問題？
13	你接手一個別人做的系統，要怎麼在半小時內判斷它的資料有沒有外洩風險？
14	你的下一個專案有一個「同時只能有一個人成功」的動作。你會怎麼做？

第 7 題和第 9 題答不出來的話，這個案例的核心沒有傳到。

常見卡關

症狀	原因	解法
`command not found: npm`	沒裝 Node.js	到 nodejs.org 下載 LTS 版本
畫面說「尚未連上資料庫」	`.env` 沒填或填錯	檢查兩個值有沒有貼完整（金鑰很長，容易少複製）
什麼資料都讀不到，但沒有錯誤	開了 RLS 但沒寫政策	**這是正常行為不是壞掉。** 回階段 6 補政策
`permission denied for table`	第一層沒開	補 GRANT
`new row violates row-level security policy`	第二層擋下	檢查「寫進去的那一列要符合什麼條件」
明明登入了卻寫不進去	規則用到的帳號資料沒有權限	本機測試的替代品要補上使用權（階段 8–1 第 4 點）
信裡的連結點下去被擋掉	Redirect URLs 沒加	Authentication → URL Configuration 補上
上線後登入壞掉	正式網址沒加進 Redirect URLs	同上，把正式網址也加進去
改了金鑰但網站還連到舊的	`PUBLIC_` 開頭的設定是打包時寫死的	重新部署一次
測試 2、5、6 出現 ✗，B 看得到別人的預約	用管理員身分測，政策被無視了	補上「換身分」那幾行
專案打不開、一直轉圈	免費專案閒置一週被暫停	到 dashboard 按恢復
資料表建了兩次	`schema.sql` 重複執行	這份 SQL 本來就設計成可以重複執行，不會有事
訪客讀私人資料回傳「空的」而不是「沒有權限」	Supabase 預設把權限給了訪客	明確收回（階段 6–1 第 4 點）。資料沒外洩，但第一層沒在守

症狀	原因	解法
想擋的操作回傳 HTTP 204，看起來像成功	204 是「跑完了，影響 0 列」	回頭查資料確認，不要看回應碼
某個區塊像是沒套到樣式	那個區塊是 JavaScript 動態產生的，Astro 的區域樣式對它無效	把那幾條樣式移到全域的 `base.css`
卡超過 20 分鐘	（無）	整段錯誤訊息貼給 Claude，再加一句：「請先不要修改任何東西。先告訴我最可能的三個原因，以及你準備先檢查哪一個。」還是不行就先跳過，每個階段的 🆘 都寫了能不能跳過

名詞白話對照表

資料庫

名詞	需要知道到什麼程度
資料庫	一個專門存資料、而且很會查資料的地方。可以想成很多張 Excel 工作表，但查詢快得多、而且能同時很多人用
資料表（table）	一張工作表。有欄位（第一列的標題），有一列一列的資料
欄位（column）	工作表的一欄，例如「開窯時間」
列（row）	工作表的一列，也就是一筆資料
SQL	跟資料庫溝通用的語言。你不用會寫，但要看得懂 AI 寫的中文註解
PostgreSQL	一套很主流的資料庫軟體。Supabase 底下就是它
schema	資料庫的結構設計：有哪幾張表、每張表有哪些欄位
約束（constraint）	寫在資料庫裡的規則，例如「同一個人同一窯只能有一筆」
unique 約束	約束的一種：「這一欄（或這幾欄的組合）不能重複」。一人一窯只能預約一次就是靠它
交易（transaction）	「這幾件事要嘛全部成功，要嘛全部不算」。轉帳是典型例子
seed（示範資料）	一批假的資料，用來讓剛建好的資料庫有東西可以測
stub（替代品）	假的、但長得一樣的東西。`_local-auth-stub.sql` 補上 Supabase 特有的兩樣東西，讓你的規則能在自己電腦上跑

權限與安全

名詞	需要知道到什麼程度
GRANT	第一層權限。決定你能不能碰某一張表
RLS（列層級安全 row level security）	第二層權限。決定你能碰這張表的哪幾列
政策（policy）	RLS 底下的一條具體規則，例如「只能看自己的預約」

名詞	需要知道到什麼程度
可以公開的金鑰	會被打包進網頁，本來就公開。它拿得到什麼，由資料庫的規則決定。新制叫 Publishable，舊制叫 anon
不能外流的金鑰	**無視所有規則**，讀走、改掉、刪光都做得到。絕對不能出現在前端。新制叫 Secret，舊制叫 service_role
通行證（JWT）	登入後拿到的一張有防偽的證明。資料庫靠它知道「現在是誰」
權限比較大的函式（security definer）	讀得到整張表，但只回傳你該知道的那個數字
縱深防禦	同一件事用兩層擋。一層寫錯了，還有另一層

前端與部署

名詞	需要知道到什麼程度
前端	使用者看得到的那一層：版面、字體、按鈕
後端	使用者看不到的那一層：資料怎麼存、誰能看什麼。**這個案例的後端就是 Supabase**
API	兩個程式之間講話的窗口。Supabase 幫你把資料表自動變成 API
靜態網站	事先做好的一堆檔案，沒有伺服器在跑。便宜、快、不容易掛
版本控制	你程式碼每一次修改的紀錄。推上 GitHub 的就是它。「`.env` 不進版本控制」＝這個檔案不會被推上去
環境變數（`.env`）	放設定值的地方，例如網址和金鑰。不會進版本控制
打包（build）	上線前的一道手續，把你寫的檔案組裝成瀏覽器看得懂的成品。`PUBLIC_` 開頭的設定值就是在這一刻被寫死進檔案裡的
部署	把做好的網站放到網路上
HTTP 狀態碼	伺服器回應的代號。2xx 代表「這個請求跑完了」，**但不代表你想做的事成功了**（見階段 12 的 204）
Vercel	一個放網站的服務，免費方案就夠這個案例用
repo	版本控制裡的一個專案。你的程式碼放在裡面，含每一次的修改紀錄
PostgREST	Supabase 用的工具，會把資料表自動變成網路上可以呼叫的窗口
GoTrue	Supabase 用來處理註冊、登入、發通行證的工具

名詞	需要知道到什麼程度
magic link（免密碼登入）	留 email、收一封信、點連結就登入了。網站裡不存密碼
終端機	用打字下指令的視窗。大部分指令由 Claude 執行。階段 8 要輸入資料庫密碼時，用的就是它

各階段預估時間

階段	主題	讀完	動手
（無）	開始之前（含操作卡）	20 分	20 分
1	先看成品	5 分	15 分
2	判斷需不需要資料庫	6 分	10 分
3	兩把金鑰與信任邊界	10 分	15 分
4	建立專案，搬進設計規範	8 分	30 分
5	設計兩張資料表	8 分	25 分
6	加上兩層權限	12 分	30 分
7	產生五筆示範窯次	4 分	10 分
8	在自己電腦上驗證規則	12 分	45 分
9	把窯次資料接到畫面上	8 分	30 分
10	做不用密碼的登入	6 分	30 分
11	做預約與取消	8 分	35 分
12	在正式環境再驗一次	10 分	30 分
13	重現超額預約	6 分	25 分
14	把檢查與寫入合成一個動作	8 分	35 分
15	用同一個方法再驗一次	4 分	20 分
16	堵住繞過網頁的那條路	6 分	25 分
17	推上 GitHub 並部署	8 分	40 分
	合計	**約 2.5 小時**	**約 7.8 小時**

時間有限的話，階段 8（本機測試）可以從 8–3 開始跳過，用階段 12 的正式環境測試代替。8–2 的測試檔還是要產生。 **不要跳過階段 6、13、15、16。** 跳掉它們，這個案例就只剩照著做一個會存資料的網站。

一次做不完很正常。建議的切法：

分段	階段	產出
第一次	開始之前 → 階段 8	資料表、權限、示範資料，以及七項全過的測試（**還沒有畫面**）
第二次	階段 9 → 階段 12	看得到、登入得了、預約得了，而且在正式環境驗過
第三次	階段 13 → 階段 17	重現漏洞、修掉、再驗一次、堵住繞過、上線

第一次做完之後畫面上還是空的。這個案例的順序刻意把規則放在畫面之前。